

Gestione Memoria Centrale

Luca Spalazzi

spalazzi@dii.univpm.it

www.univpm.it/luca.spalazzi



Dipartimento di Ingegneria
dell'Informazione



cini
Cyber Security Lab@UnivPM

Gruppo Ingegneria Informatica
@UnivPM



- **Introduzione**
 - Codice rilocabile
 - Swapping
- **Gestione della Memoria**
 - Allocazione contigua
 - Paginazione
 - Segmentazione
- **Memoria Virtuale**
 - Paginazione su richiesta
 - Algoritmi di sostituzione delle pagine: FIFO, Ottimale, LRU, Orologio
 - Algoritmi di assegnazione dei frame liberi: Uniforme, Proporzionale, per Priorità, basata sul Working Set, basata su Page Fault Rate
- **Gestione Memoria Multiprocessori**
- **Gestione della Memoria in Linux**
 - Memoria Fisica
 - Memoria Virtuale



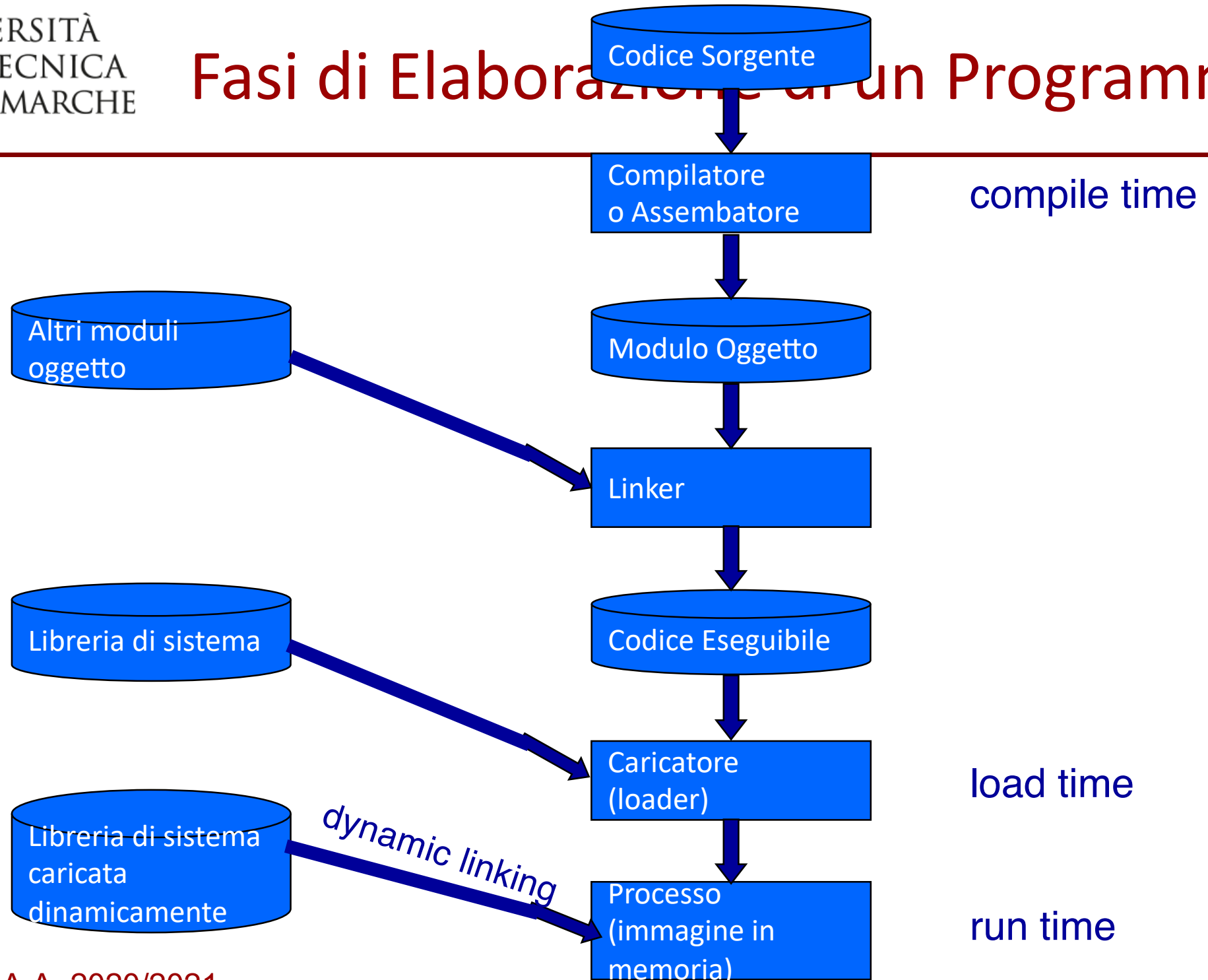
UNIVERSITÀ
POLITECNICA
DELLE MARCHE

Memoria

INTRODUZIONE

- Un programma in genere risiede in un disco sotto forma di file binario eseguibile.
- Il programma per essere eseguito deve essere caricato in memoria ed inserito all'interno di un processo.
- **Coda di Ingresso (input queue)** – insieme di processi che devono essere caricati in memoria per essere eseguiti (se è presente lo scheduler a lungo termine, la coda viene gestita dallo scheduler a lungo termine).
- I programmi utente passano attraverso diversi passi prima di poter essere eseguiti.

Fasi di Elaborazione di un Programma



Binding di Istruzioni e Dati a Indirizzi di Memoria

Può avvenire in una qualsiasi delle seguenti fasi.

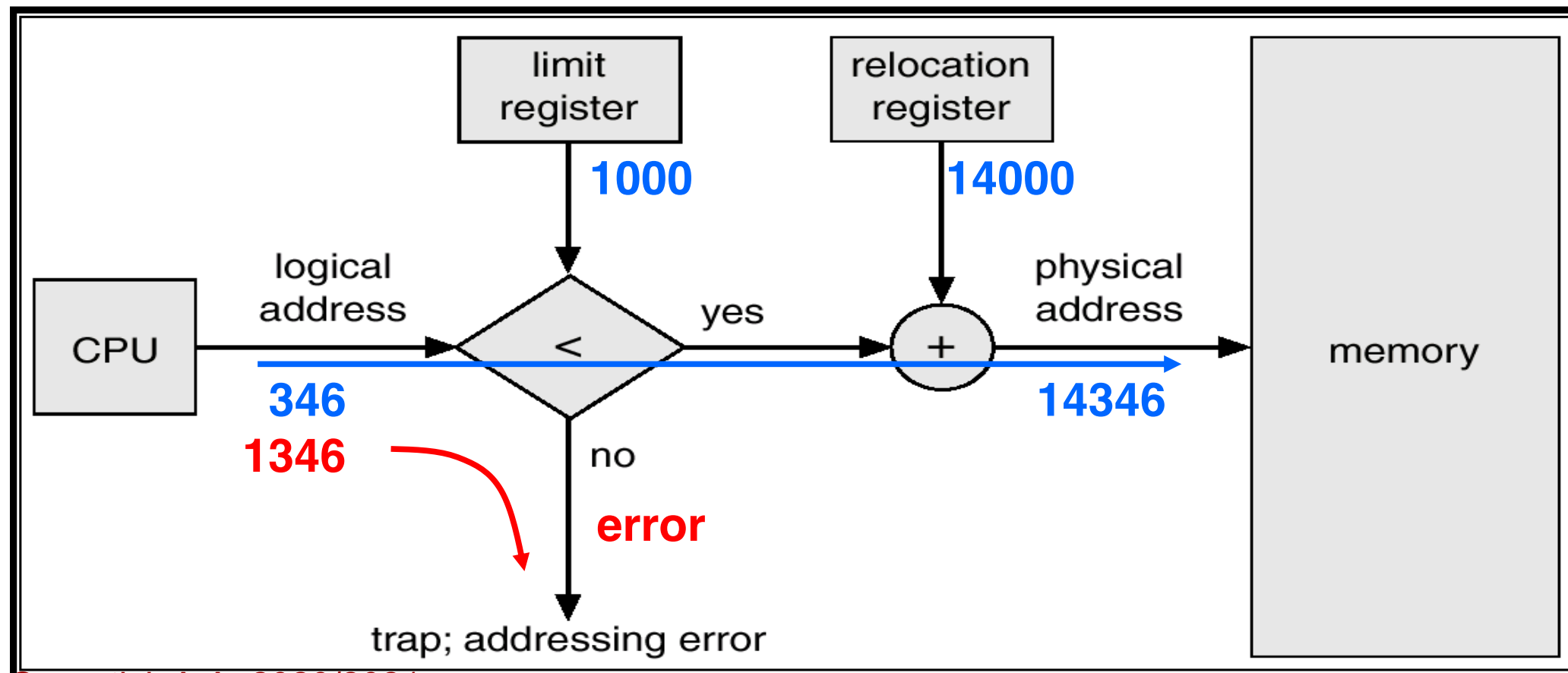
- **Compilazione:** Se è noto a priori dove il processo risiederà in memoria, può essere generato un **codice assoluto**, che contiene direttamente **indirizzi assoluti**; se la locazione di inizio cambia, bisogna ricompilare il programma. Un esempio sono i **.com** di MS-DOS
- **Caricamento:** Se dove il processo risiederà in memoria NON è noto a priori, il compilatore genera un **codice rilocabile**, che contiene **indirizzi rilocabili**. È compito del caricatore fare il binding tra indirizzi rilocabili e gli indirizzi assoluti (**rilocazione statica**).
- **Esecuzione:** Se durante l'esecuzione il processo può essere spostato da un segmento di memoria ad un altro, il binding deve essere ritardato fino al momento dell'esecuzione (**rilocazione dinamica**). È necessario dell'hardware di supporto per gestire questa situazione (per esempio i registri base e limite). La maggior parte dei SO d'uso generale impiega questo metodo

Spazio di Indirizzi Logici e Fisici

- Per una corretta gestione della memoria abbiamo bisogno di due tipi di spazio di indirizzi separati tra loro, anche se esiste un legame che li unisce.
 - **Indirizzi Logici (Indirizzi Virtuali)** – sono gli indirizzi generati dalla CPU, cioè quelli che vengono messi in R_{lp}, il registro indice della CPU, e trasmessi attraverso il bus indirizzi.
 - **Indirizzi Fisici** – sono gli indirizzi visti dalla unità di memoria, cioè quelli che vengono messi in R_{lm}, il registro indice della Memoria (anche chiamato MAR – Memory Address Register).
- Indirizzi logici e fisici coincidono quando il binding viene fatto in fase di compilazione o caricamento, sono diversi quando il binding viene fatto in fase di esecuzione
- Il programma utente ha a che fare con gli indirizzi logici e non vede mai gli indirizzi fisici.
- L'hardware che traduce gli indirizzi logici in fisici è il **Memory Management Unit** (MMU).

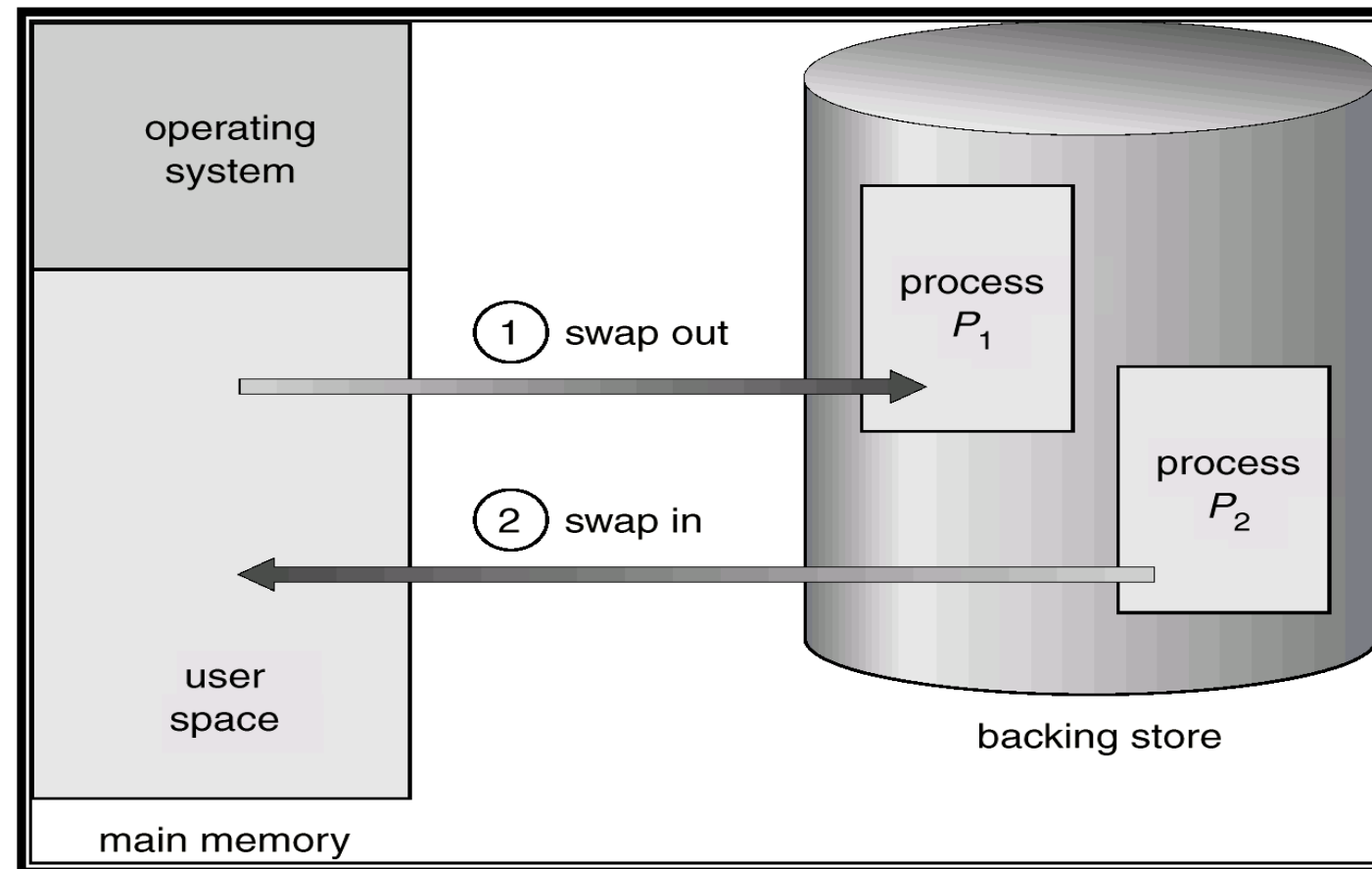
Memory-Management Unit (MMU)

- Usa il **registro di rilocalizzazione** (al posto del registro base). Il suo valore viene sommato a tutti gli indirizzi generati dalla CPU.
- N.B. l'uso dei registri base è diverso, infatti il registro base viene confrontato con (e NON sommato a) l'indirizzo generato dalla CPU.



Swapping (Avvicendamento)

- Un processo può che si trova in memoria centrale, può essere temporaneamente trasferito nella memoria secondaria, per far posto in memoria centrale ad un altro processo. Successivamente, quando deve riprendere la sua esecuzione, viene riportato in memoria centrale.
- Il tempo di swapping è un fattore critico.





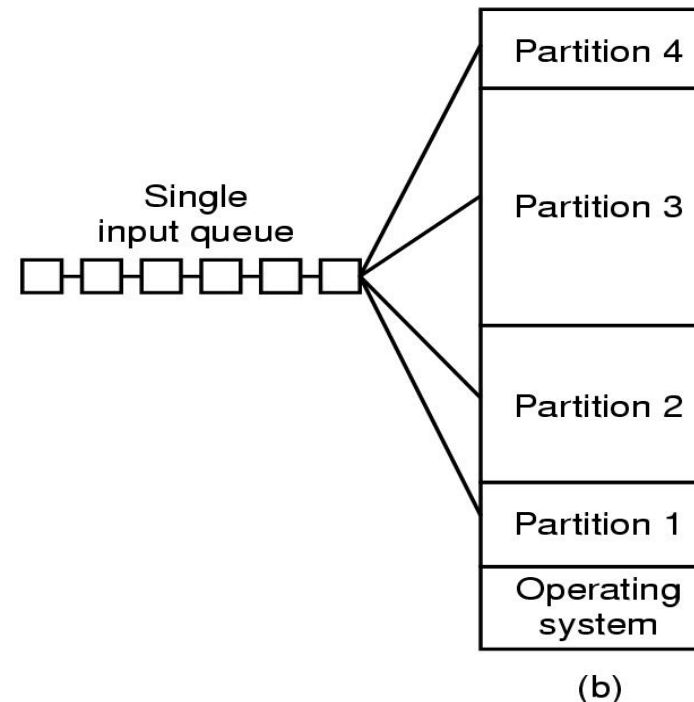
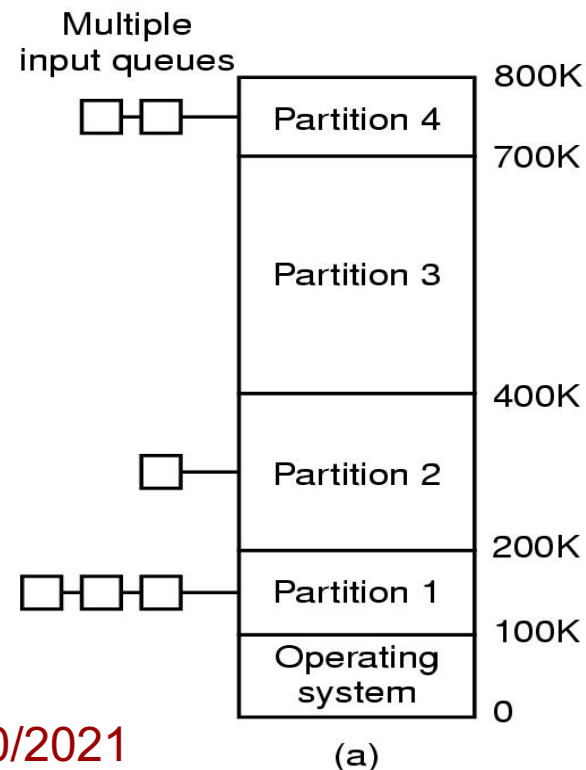
Memoria

GESTIONE DELLA MEMORIA

- ✓ Introduzione
 - ✓ Codice rilocabile
 - ✓ Swapping
- Gestione della Memoria
 - Allocazione contigua
 - Paginazione
 - Segmentazione
- Memoria Virtuale
 - Paginazione su richiesta
 - Algoritmi di sostituzione delle pagine: FIFO, Ottimale, LRU, Orologio
 - Algoritmi di assegnazione dei frame liberi: Uniforme, Proporzionale, per Priorità, basata sul Working Set, basata su Page Fault Rate
- Gestione Memoria Multiprocessori
- Gestione della Memoria in Linux
 - Memoria Fisica
 - Memoria Virtuale

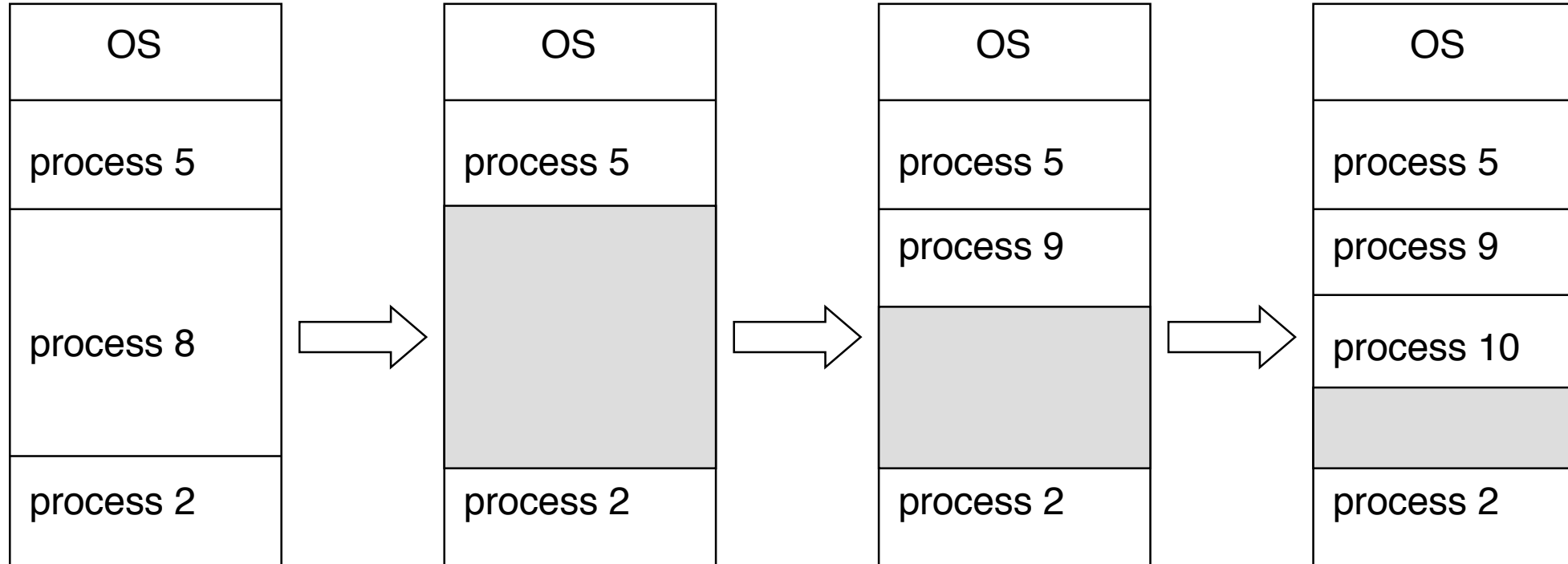
Allocazione Contigua – Partizioni Fisse

- La memoria viene suddivisa in un certo numero di partizioni di dimensione fissa.
- Ogni processo viene caricato in una partizione.
- Quando un processo termina l'esecuzione, libera una partizione che può essere occupata da un altro processo in attesa nella **coda di ingresso** (**singola** o **multipla**).
- Il grado di multiprogrammazione è limitato dal numero di partizioni



Allocazione Contigua – Partizioni Variabili

- All'inizio ci sono due partizioni, una allocata al SO e l'altra (il resto della memoria) è libera.
- Il SO conserva in una tabella le **partizioni libere** (buchi) e le **partizioni allocate**, ognuna con dimensioni diverse dalle altre.
- Quando arriva un processo, viene allocato in un buco abbastanza grande da contenerlo. La parte di buco non utilizzata viene lasciata come partizione libera per un altro processo.



Allocazione Contigua – Partizioni Variabili

Problema: come soddisfare una richiesta di dimensione n avendo a disposizione una lista di buchi.

- **First-fit:** Alloca il primo buco di dimensioni sufficienti.
- **Best-fit:** Alloca il buco più piccolo in grado di soddisfare la richiesta; bisogna scandire l'intera lista (a meno che non sia ordinata per dimensione). Produce le parti di buco inutilizzate più piccole.
- **Worst-fit:** Alloca il buco più grande; bisogna scandire l'intera lista (a meno che non sia ordinata per dimensione). Produce le parti di buco inutilizzate più grandi.



First-fit e best-fit sono migliori di worst-fit in termini di velocità e utilizzo della memoria.

Allocazione Contigua – Partizioni Variabili

In memoria sono presenti i seguenti spazi liberi: **100K**, **500K**, **200K**, **300K**, **600K** (ordinati in base all'indirizzo di partenza). Come verrebbero allocati i seguenti processi:

P1 (212K), P2 (417K), P3 (112K), P4 (426K),

utilizzando gli algoritmi First Fit, Best Fit e Worst Fit?

First Fit: **P1 in 500K (rimane 288K), P2 in 600K (rimane 183K)**
 P3 in 288K (rimane 176K), P4 deve attendere

Best Fit: **P1 in 300K (rimane 88K), P2 in 500K (rimane 83K)**
 P3 in 200K (rimane 88K), P4 in 600K (rimane 174K)

Worst Fit: **P1 in 600K (rimane 388K), P2 in 500K (rimane 83K)**
 P3 in 388K (rimane 276K), P4 deve attendere

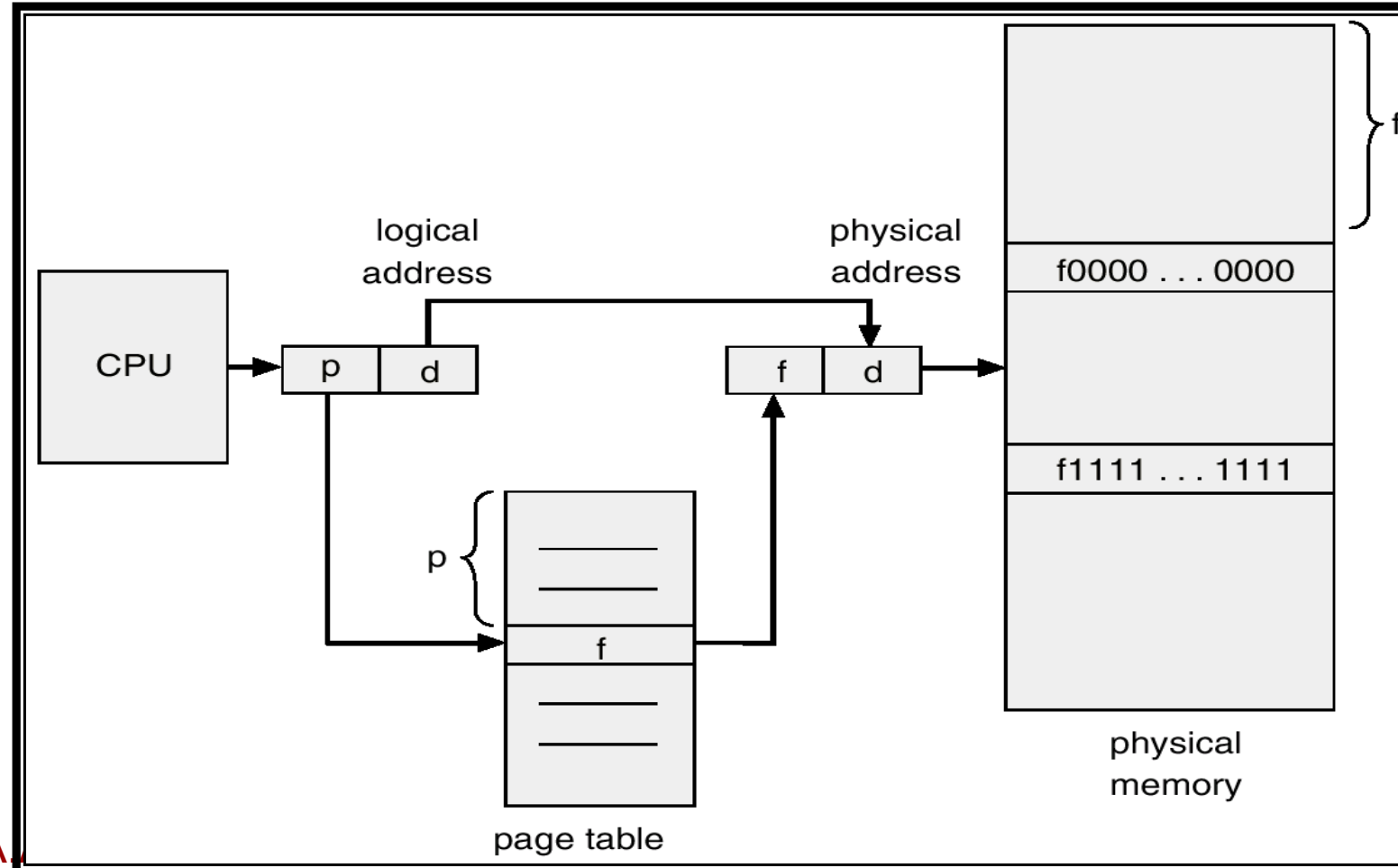
- **Frammentazione Interna** – la memoria allocata può essere più grande di quella che effettivamente serve; la memoria in più è interna alla partizione, ma non può essere usata.
 - Tipica delle **partizioni fisse**.
- **Frammentazione Esterna** – la somma totale di tutti gli spazi liberi è sufficiente per contenere un nuovo processo, ma il processo non può essere caricato in memoria perché questi spazi non sono contigui.
 - Tipica delle **partizioni variabili**.
 - La frammentazione esterna può essere eliminata con la **compattazione**.
 - Riordina il contenuto della memoria per riunire la memoria libera in un unico buco.
 - La compattazione è possibile solo se la rilocalizzazione è dinamica.
 - La compattazione è costosa.

- Lo spazio degli indirizzi logici può non essere contiguo.
- La memoria fisica è divisa in blocchi di dimensioni fisse chiamati **pagine fisiche** o **frame** (le dimensioni sono potenze di 2, tra 512 byte e 8192 byte).
- La memoria logica è divisa in blocchi chiamati **pagine logiche** o **pagine**, le cui dimensioni sono pari a quelle dei frame.
- Bisogna tenere traccia di tutti i frame liberi.
- Per eseguire un programma di dimensioni pari ad n pagine, bisogna trovare n frame liberi e caricare le pagine in quei frame.
- Viene usata una **tabella delle pagine** (di solito una per processo) per tradurre gli indirizzi logici in indirizzi fisici.
- Presenta il problema della **frammentazione interna**.

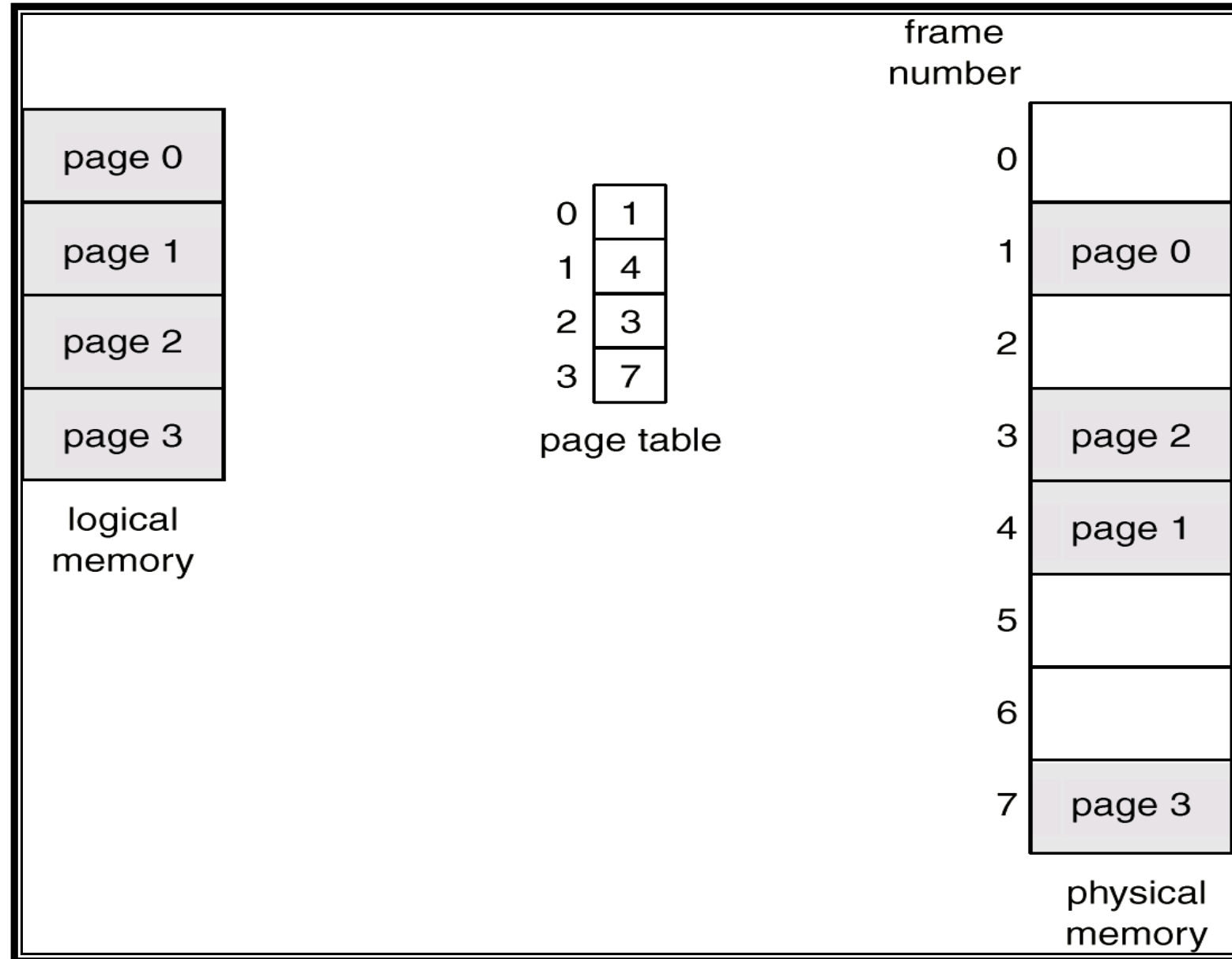
Paginazione - Traduzione degli Indirizzi

Gli indirizzi generati dalla CPU si dividono in due parti:

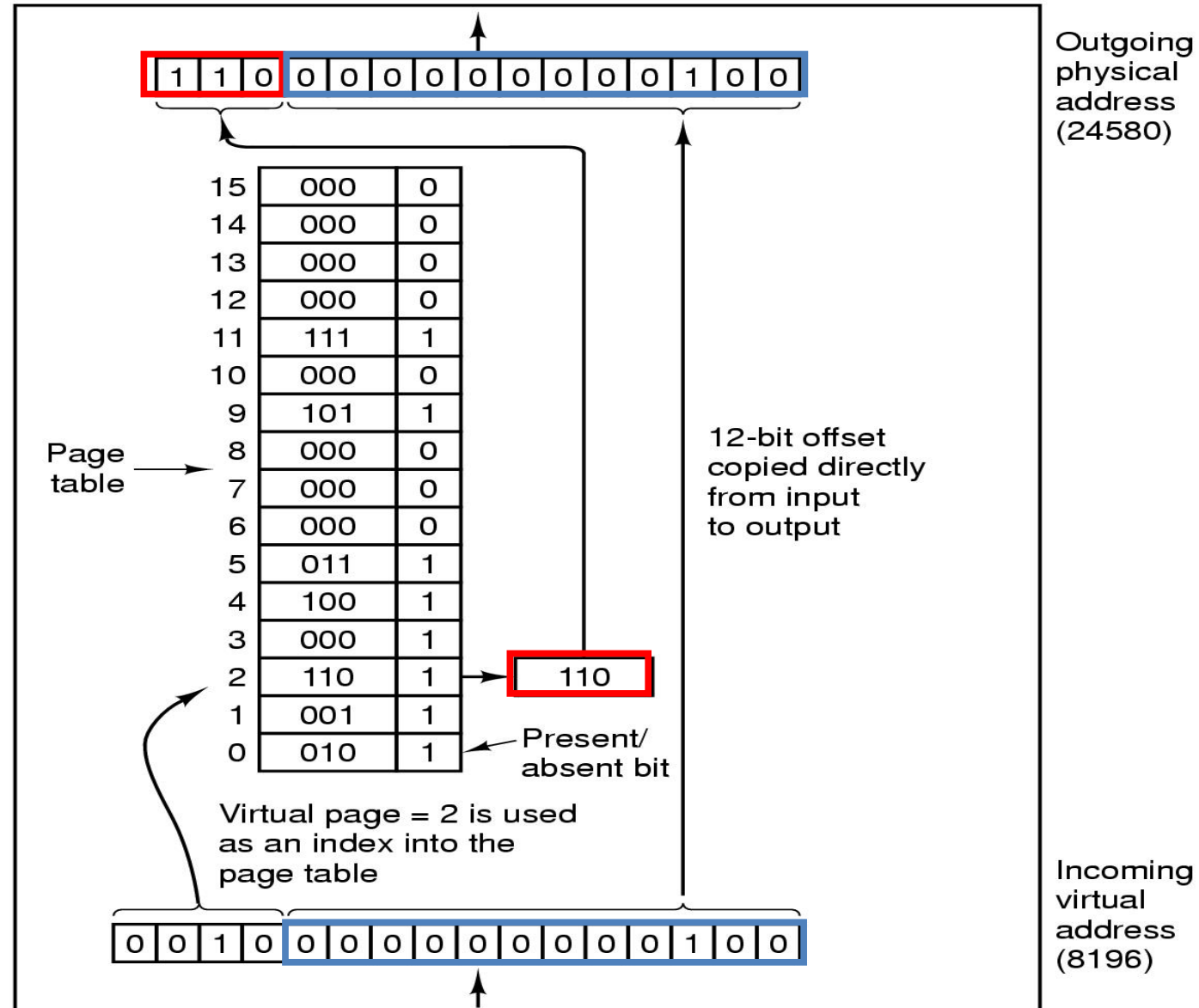
- **Numero di pagina** (p) – viene usato come indice della tabella delle pagine che contiene l'indirizzo base di ogni pagina nella memoria fisica.
- **Offset** (d) – è lo scostamento all'interno della pagina.



Paginazione - Esempio



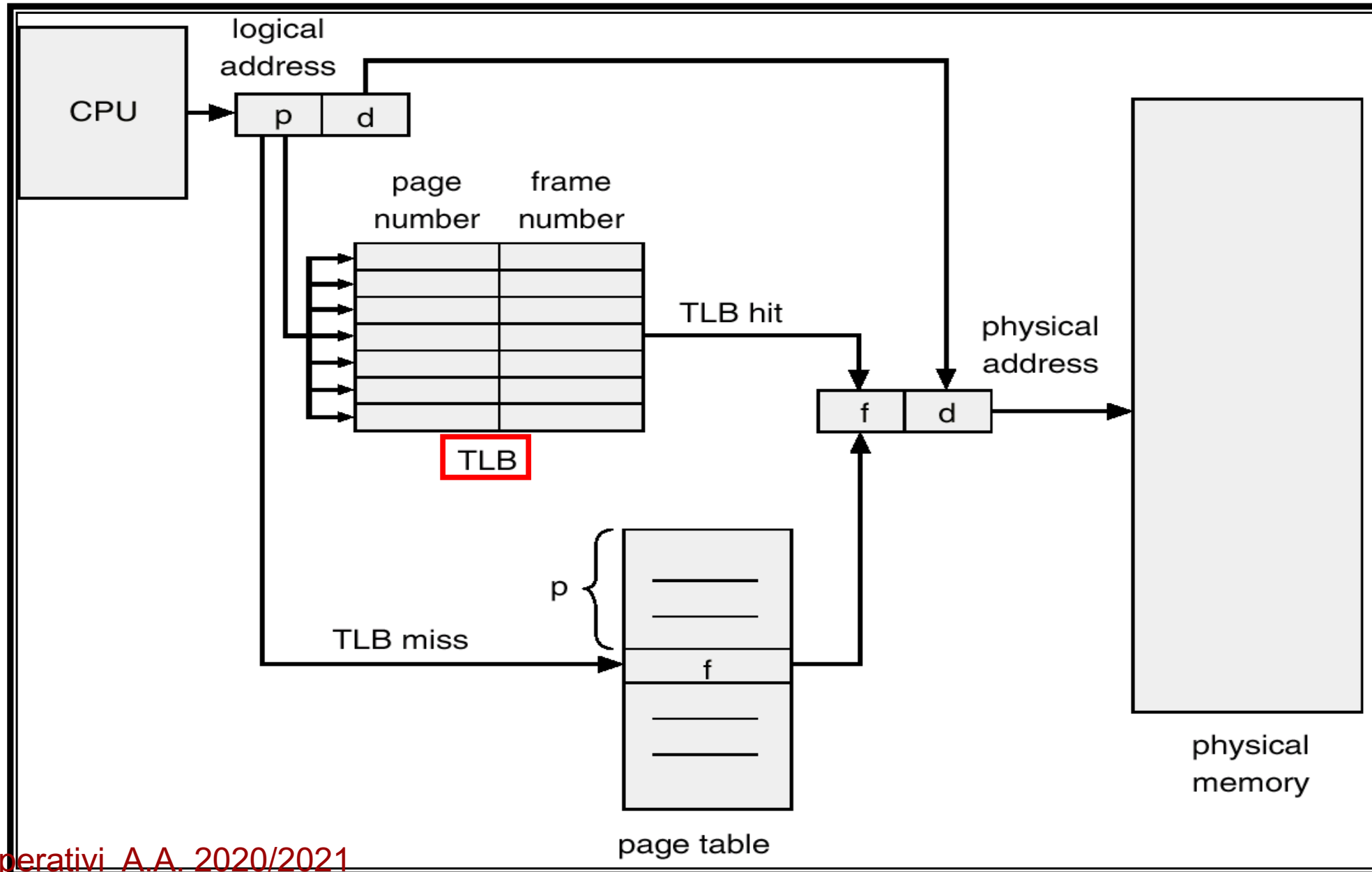
Paginazione - Esempio



Realizzazione della Tabella delle Pagine

- La Tabella delle Pagine è mantenuta in memoria.
- **Page-table base register (PTBR)** indica l'inizio della Tabella delle Pagine.
- **Page-table length register (PRLR)** indica le dimensioni della Tabella delle Pagine.
- Con questo schema, per ogni accesso a dati/istruzioni ci sono due accessi in memoria: uno alla Tabella delle Pagine ed uno ai dati/istruzioni veri e propri.
- Il problema del duplice accesso in memoria può essere risolto con una speciale memoria associativa usata come cache. Questa memoria viene chiamata **translation look-aside buffer (TLB)**
- La memoria associativa permette di cercare in parallelo su tutti gli elementi quale sia uguale ad un valore dato.

Realizzazione della Tabella delle Pagine



Realizzazione della Tabella delle Pagine

```
1.  MARMMU ← ⟨p,d⟩           // Indirizzo logico dalla CPU
2.  if (p < PTLR)               // Indirizzo interno allo spazio del processo
3.  {
4.      if (p in TLB)           // TLB hit
5.          f ← TLB(p)
6.      else
7.          f ← M[⟨PTBR,p⟩]     // TLB miss – accesso alla Tabella delle pagine
8.          MDRMMU ← M[⟨f,d⟩]  // accesso in lettura
9.                               // accesso in scrittura M[⟨f,d⟩] ← MDRMMU
10. }
11. else
12.     trap                     // Indirizzo esterno allo spazio del processo
```

Tempo Effettivo di Accesso con TLB

- Tempo di accesso alla memoria associativa = ε unità di tempo
- Tempo di accesso in memoria = 1 unità di tempo
- Hit ratio = α
percentuale di successo nel trovare l'elemento cercato nel TLB.

- **Tempo Effettivo di Accesso (EAT)**

$$\begin{aligned} \text{EAT} &= (1 + \varepsilon) \alpha + (2 + \varepsilon)(1 - \alpha) = \\ &= 2 + \varepsilon - \alpha \end{aligned}$$

- **Esempio:** tempo di accesso in memoria = 10 ns
tempo di accesso alla TLB = 1 ns
hit ratio = 0.99
 $\varepsilon = 0.1$
 $\alpha = 0.99$

$$\text{EAT} = 1,11$$

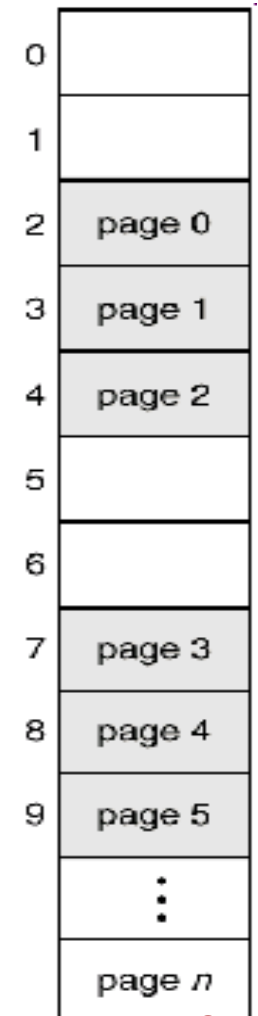
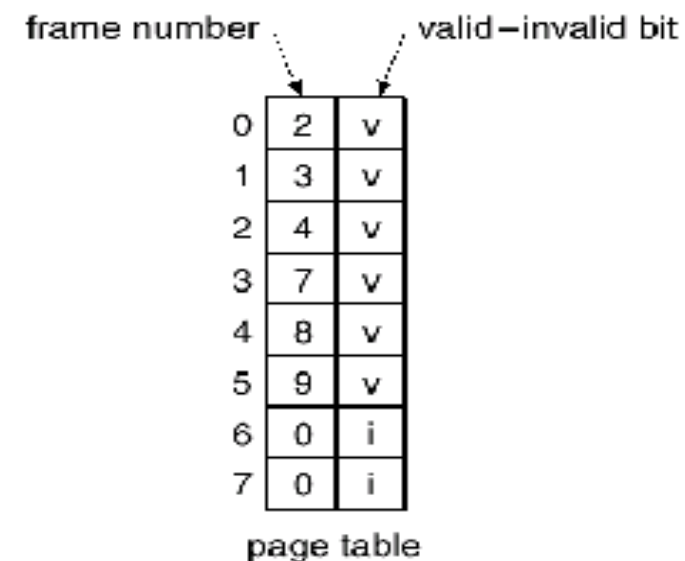
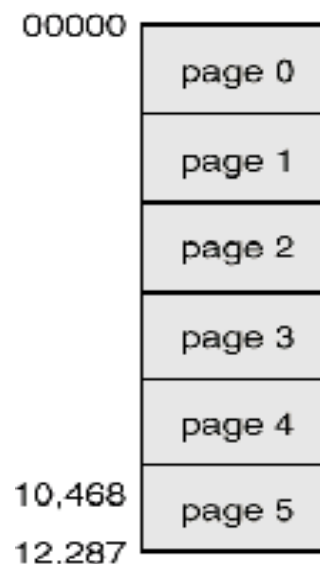
rallentamento dell'11%

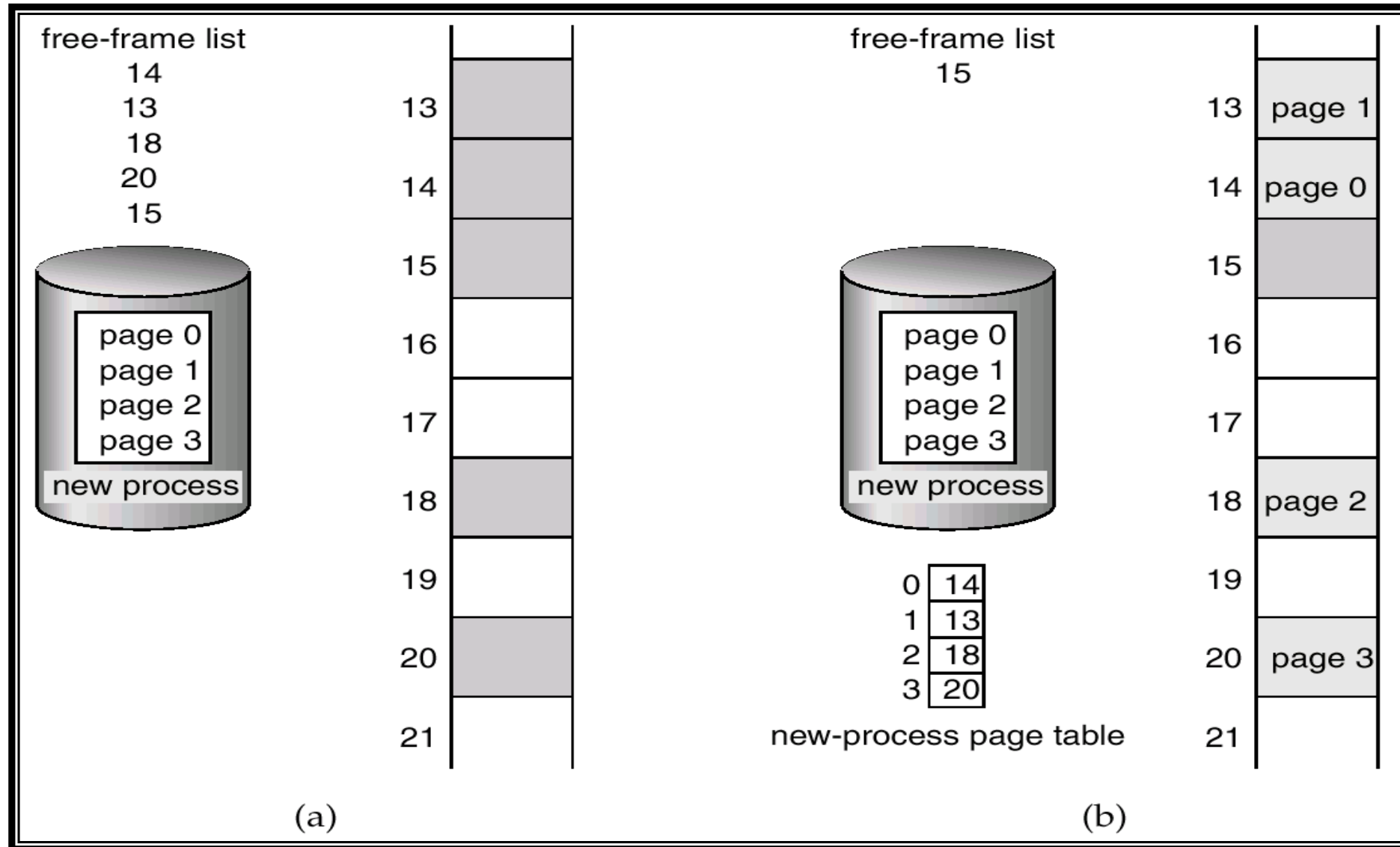
Protezione della Memoria

- Ad ogni frame viene associato un **bit di protezione**.
 - “protetto” indica che il processo può accederci solo in lettura.
 - “non protetto” indica che il processo può accederci sia in lettura sia in scrittura.
- Ad ogni elemento della Tabella delle Pagine viene associato un **bit di validità**:
 - “valido” indica che la pagina associata rientra nello spazio degli indirizzi logici del processo.
 - “non valido” indica che la pagina associata NON rientra nello spazio degli indirizzi logici.

In alternativa si può usare il **PTLR** (e il bit di validità più avanti lo useremo per altro)

- Ogni violazione produce una **trap**





Paginazione a Due Livelli

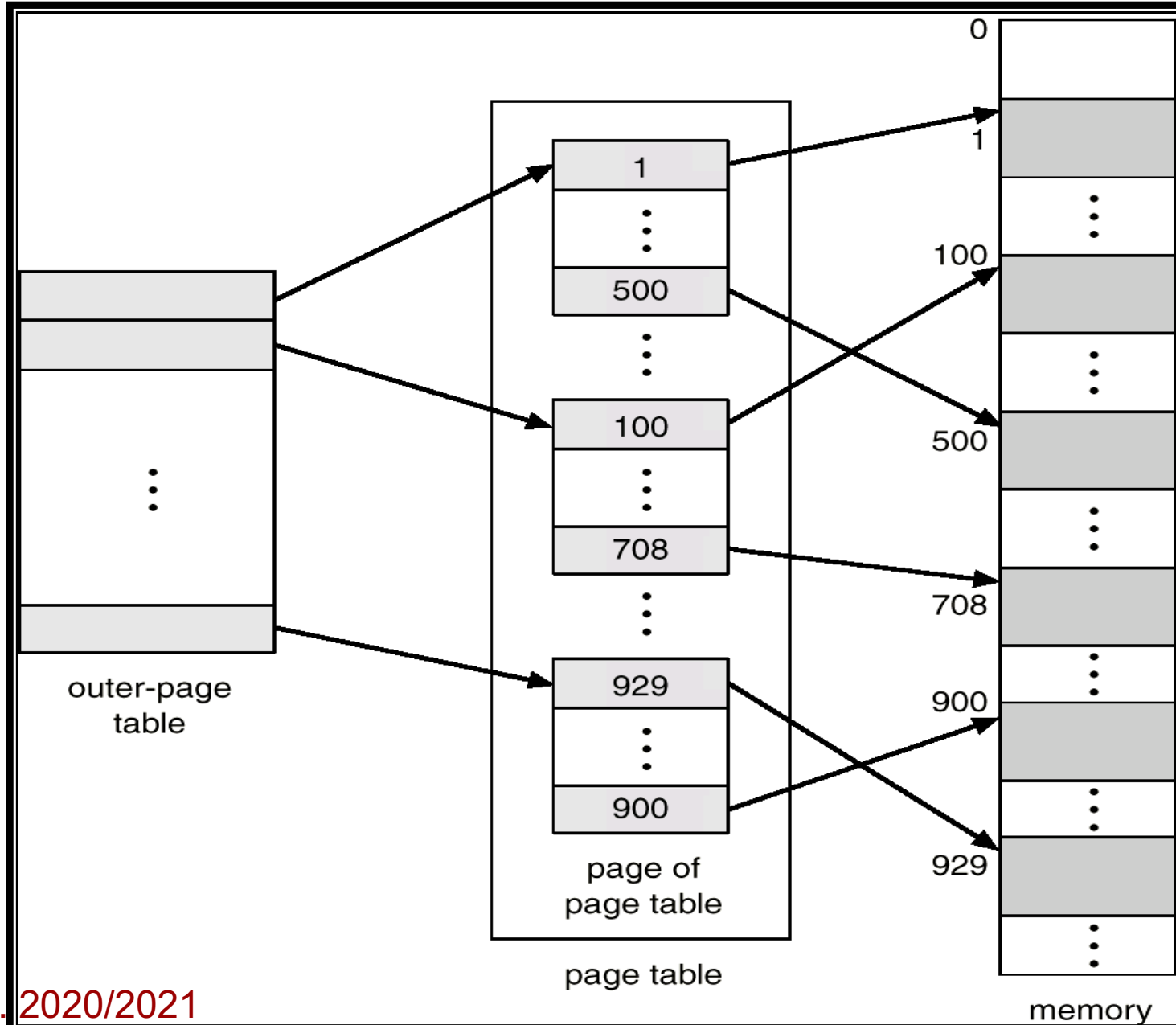
- Tecnica di paginazione per ridurre le dimensioni delle pagine. È comune in macchine con indirizzi a 32 bit o meno. Consiste nel paginare la stessa tabella delle pagine
- Su una macchina a 32-bit con pagine di 4K, l'indirizzo è suddiviso in: :
 - Un numero di pagina di 20 bit.
 - Un offset di 12 bit.
- Siccome la tabella delle pagine è paginata, il numero di pagina è ulteriormente suddiviso in:
 - Un numero di pagina di 10-bit.
 - Un offset di 10-bit.
- Quindi un indirizzo logico ha la seguente struttura:

page number		page offset
p_1	p_2	d
10	10	12

dove p_1 è l'indice della tabella esterna delle pagine, e p_2 è lo scostamento all'interno della pagina della tabella delle pagine.

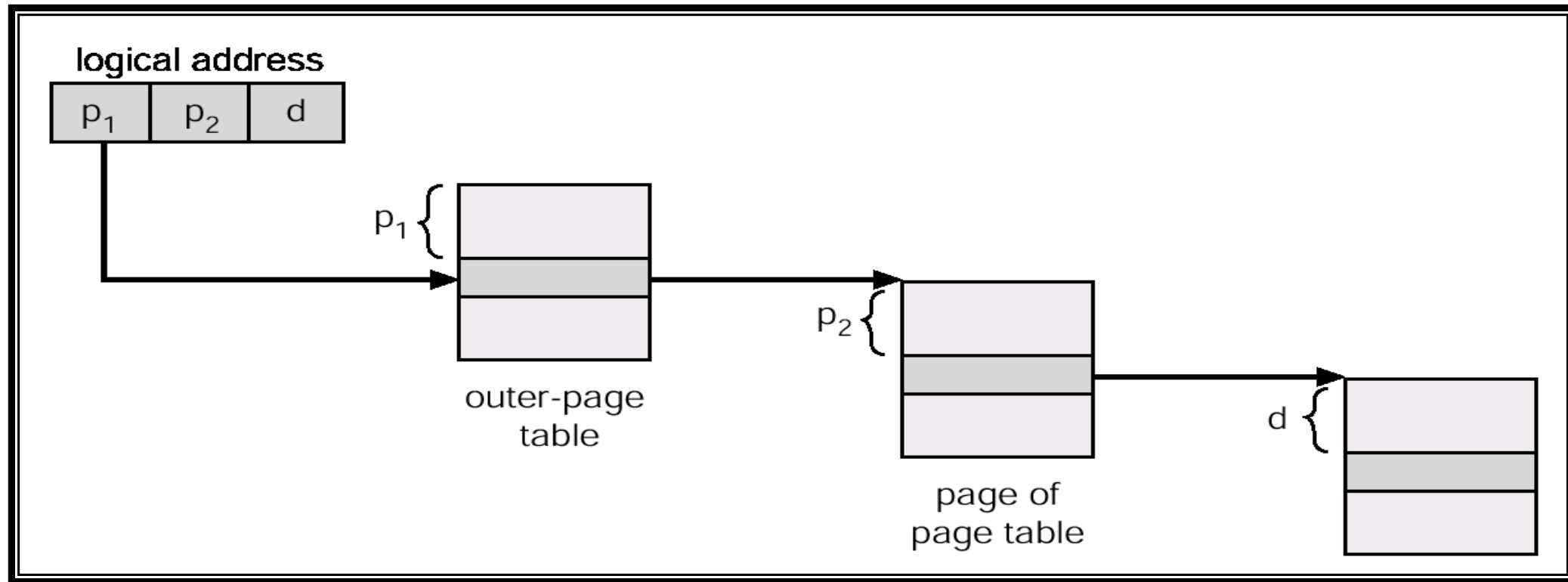
- Ad esempio il Pentium II usa questa tecnica.
- Macchine con indirizzi più lunghi di 32 bit adottano altre tecniche.

Paginazione a Due Livelli



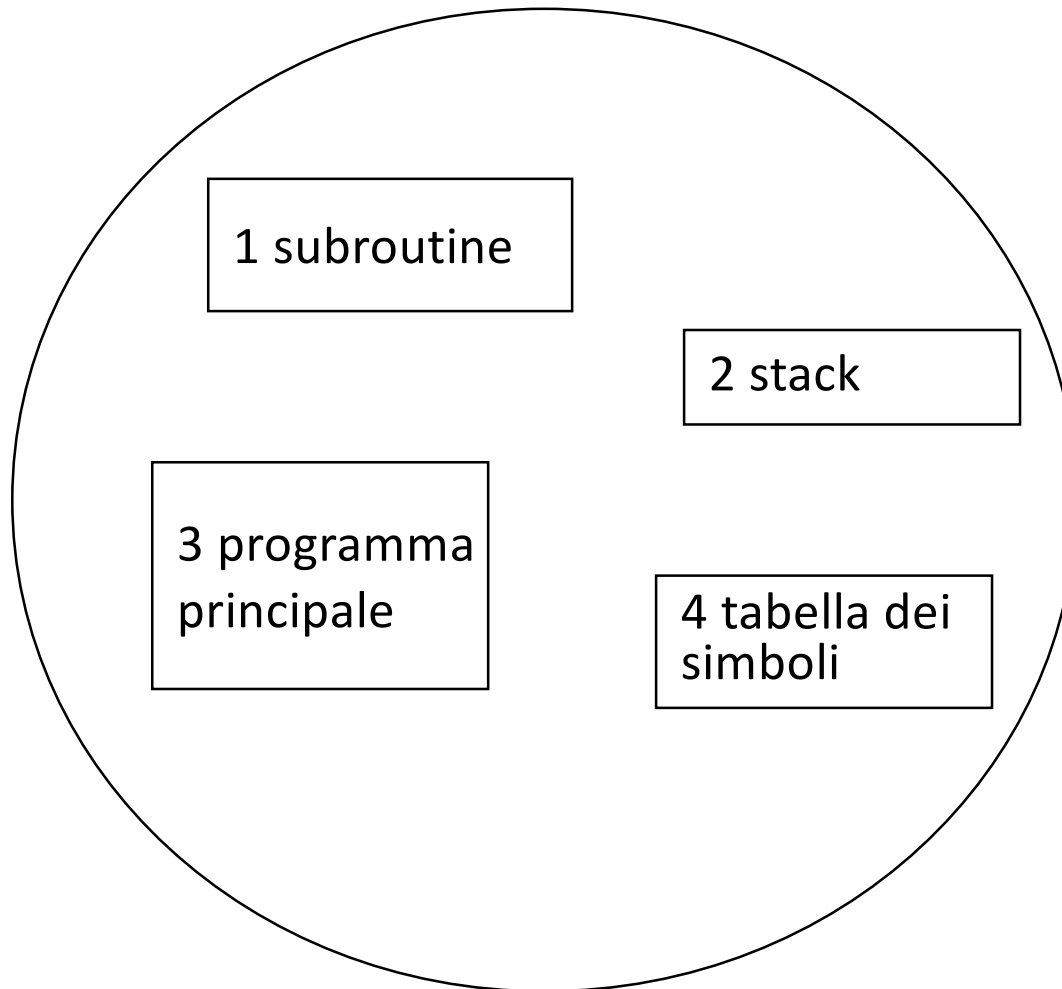
Paginazione a Due Livelli

Schema di Traduzione degli Indirizzi

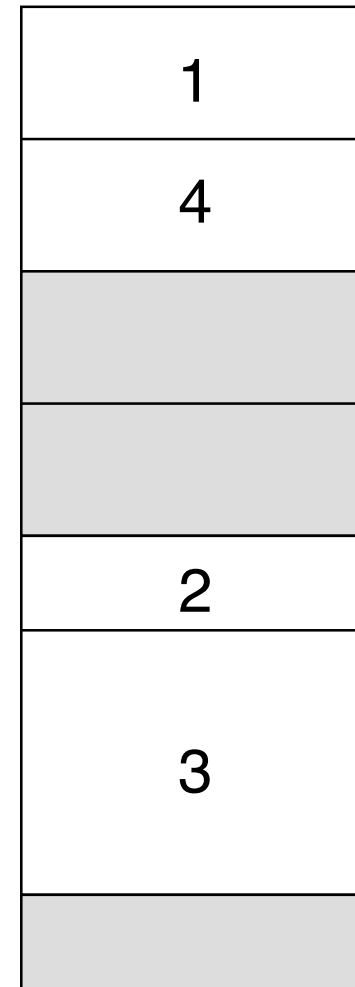


- È uno schema di gestione della memoria che si avvicina al modo in cui l'utente vede la memoria.
- Un programma è una collezione di segmenti. Ogni segmento è una unità logica. Ad esempio abbiamo:
 - programma principale,
 - procedure,
 - funzioni,
 - metodi,
 - oggetti,
 - variabili locali e variabili globali,
 - blocco comune,
 - stack,
 - symbol table, array

Segmentazione



Spazio logico degli indirizzi
(vista dell'utente)



Spazio fisico degli indirizzi
(memoria fisica)

- Gli indirizzi logici sono costituiti da una coppia:
 <numero-di-segmento, offset>,
- **Tabella dei segmenti** – permette la traduzione dallo spazio bidimensionale degli indirizzi logici allo spazio unidimensionale degli indirizzi fisici. Ogni elemento della tabella ha:
 - base – contiene l'indirizzo fisico di inizio dell'area di memoria dove risiede il segmento.
 - limite – indica la lunghezza del segmento.
- **Segment-table base register (STBR)** punta alla locazione di memoria dove risiede la tabella dei segmenti.
- **Segment-table length register (STLR)** indica il numero di segmenti; numero-di-segmento s è legale se $s < \text{STLR}$.

- Protezione.

Ad ogni elemento della tabella vengono associati:

- Bit di validità (quando = 0 $\Rightarrow$ segmento illegale)
- Privilegi di lettura/scrittura/esecuzione

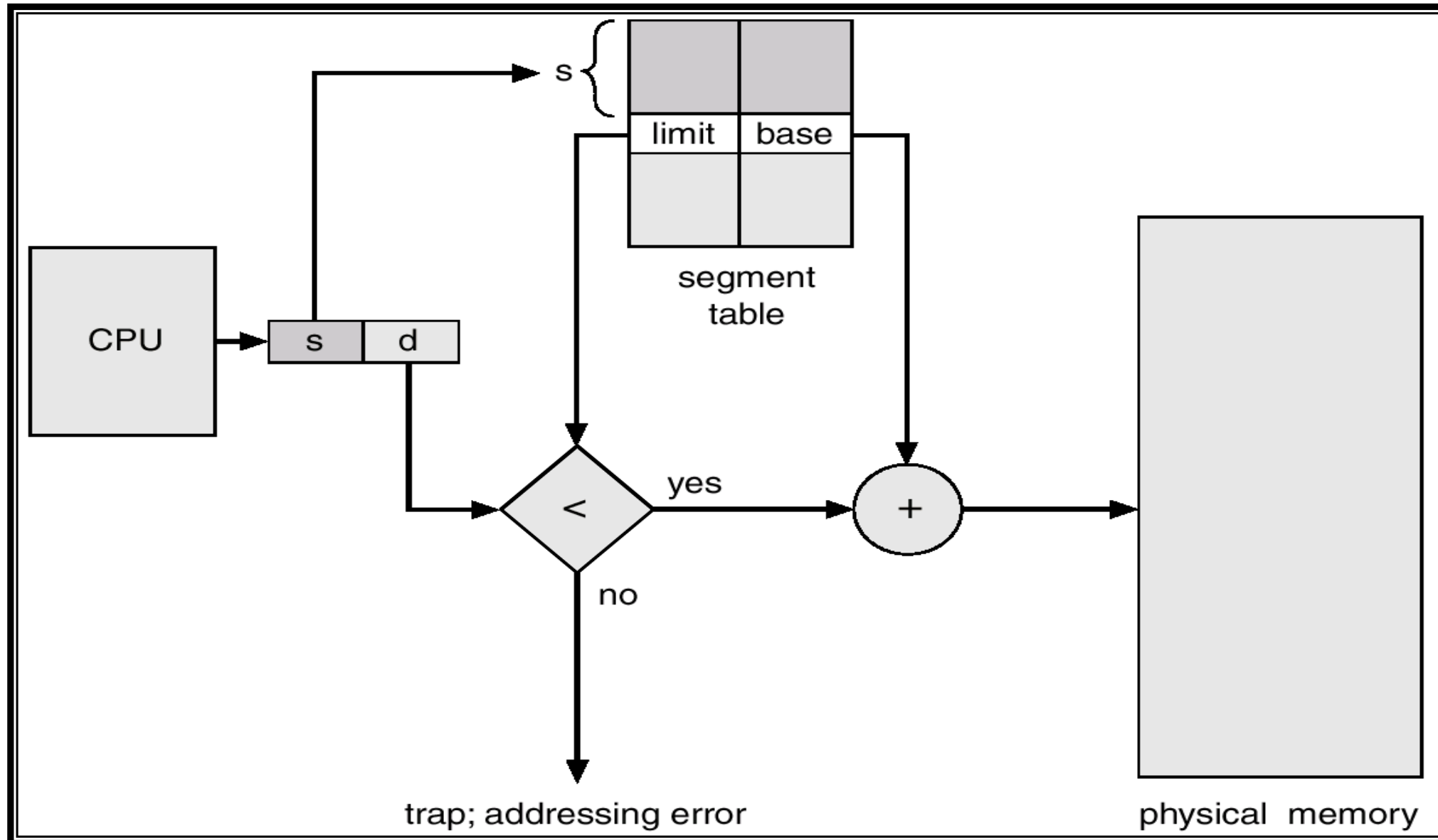
- Condivisione dei segmenti.

Siccome la protezione avviene a livello di ogni singolo segmento, i segmenti possono essere condivisi tra processi diversi.

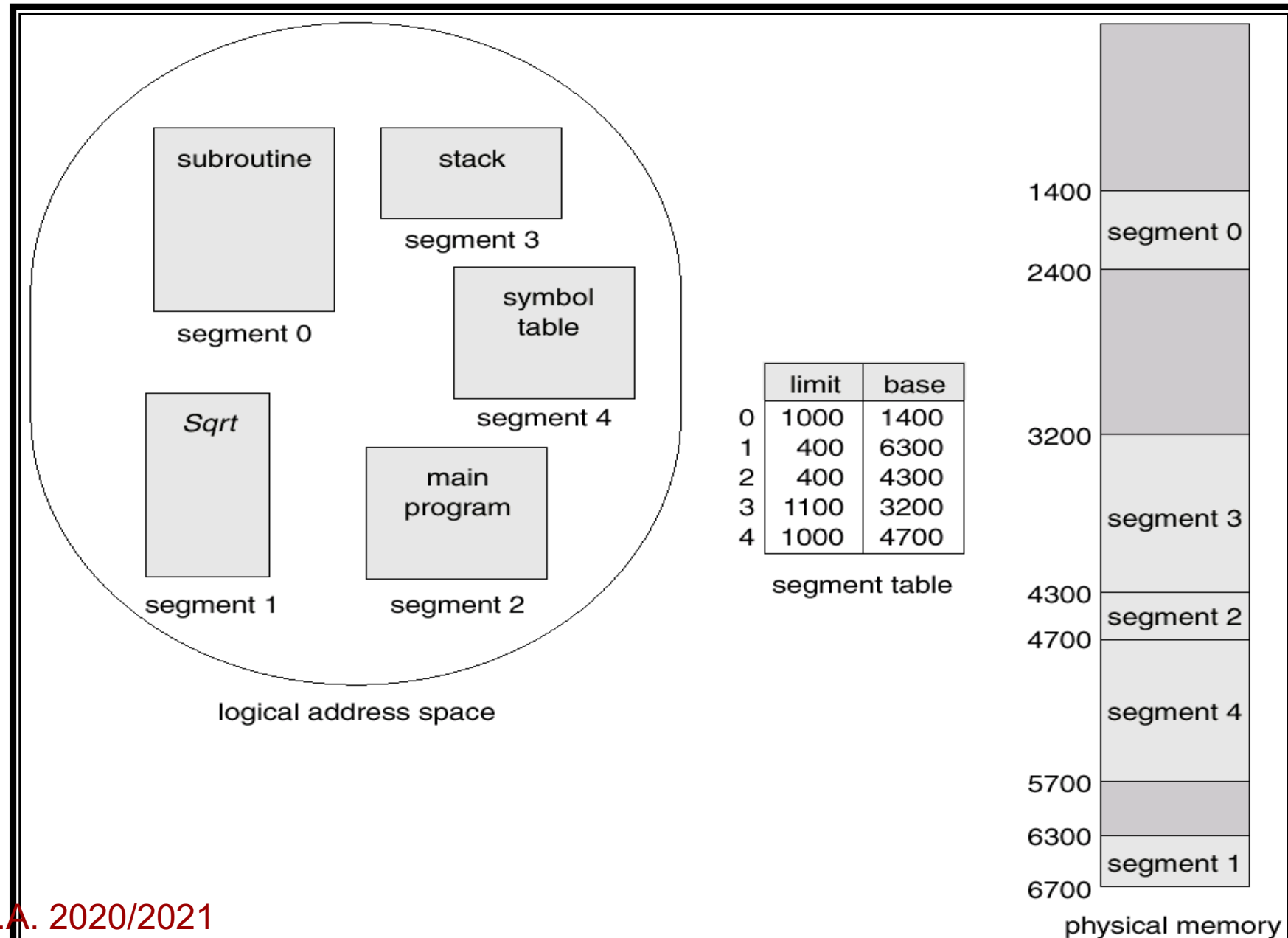
- Problema di allocazione dinamica della memoria.

Siccome i segmenti sono di lunghezza variabile, si presentano gli stessi problemi affrontati per l'allocazione contigua a partizioni variabili (frammentazione esterna, first/best/worst fit).

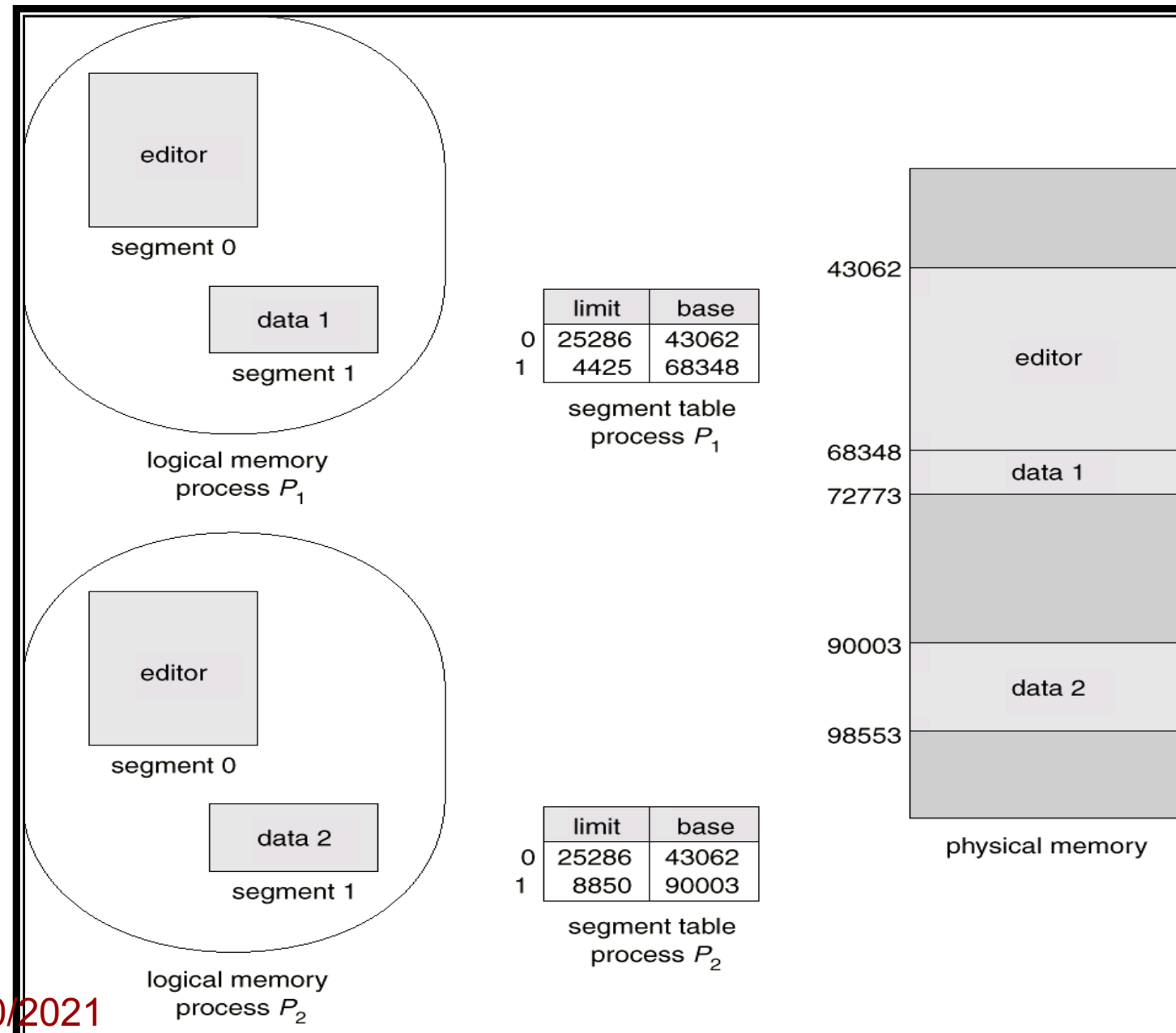
Segmentazione



Segmentazione - Esempio



Condivisione di Segmenti





Memoria

MEMORIA VIRTUALE

✓ Introduzione

- ✓ Codice rilocabile
- ✓ Swapping

✓ Gestione della Memoria

- ✓ Allocazione contigua
- ✓ Paginazione
- ✓ Segmentazione

■ Memoria Virtuale

- Paginazione su richiesta
- Algoritmi di sostituzione delle pagine: FIFO, Ottimale, LRU, Orologio
- Algoritmi di assegnazione dei frame liberi: Uniforme, Proporzionale, per Priorità, basata sul Working Set, basata su Page Fault Rate

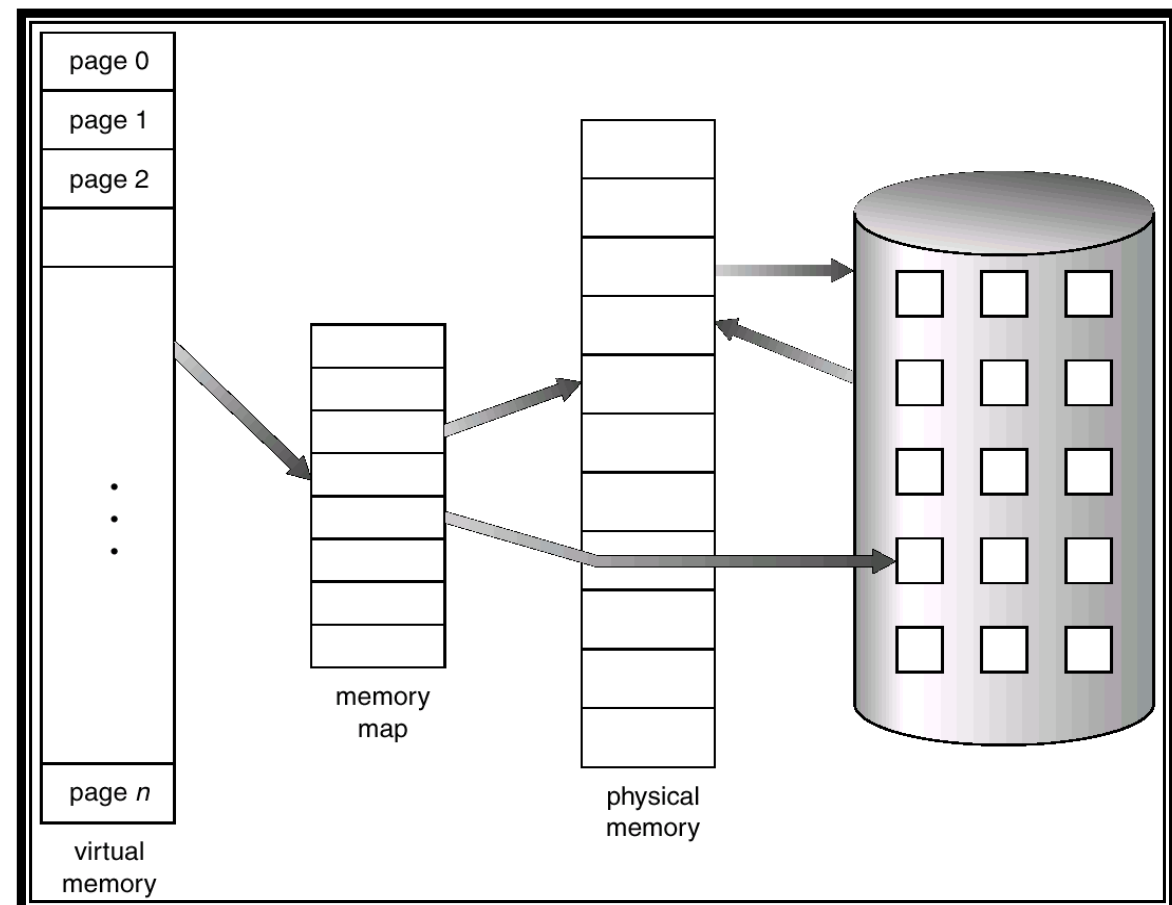
■ Gestione Memoria Multiprocessori

■ Gestione della Memoria in Linux

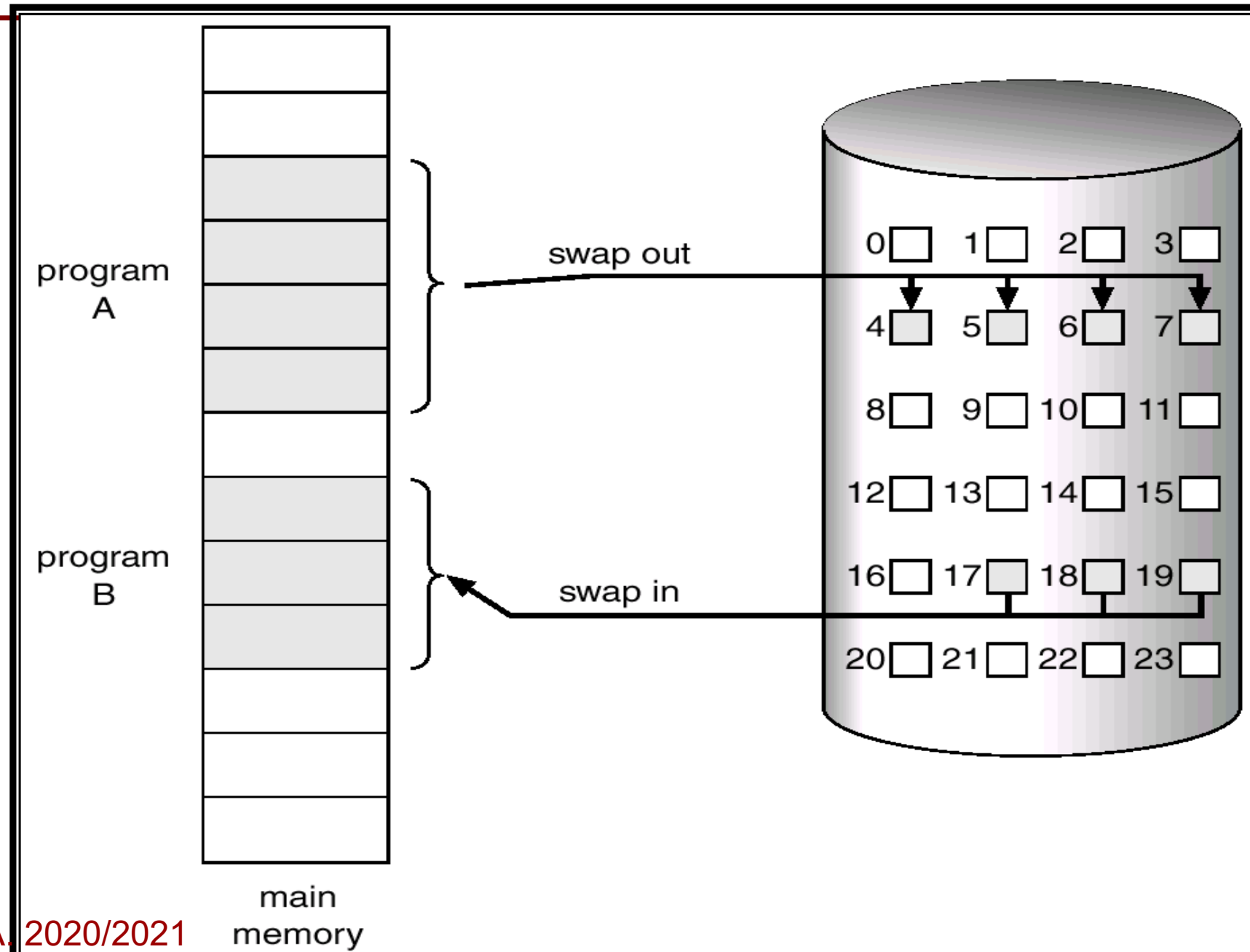
- Memoria Fisica
- Memoria Virtuale

Memoria Virtuale

- Lo spazio degli indirizzi logici è più grande dello spazio degli indirizzi fisici.
 - Solo la porzione di processo necessaria viene tenuta in memoria per l'esecuzione.
 - Lo spazio degli indirizzi può essere condiviso tra processi diversi.
 - La creazione dei processi è più efficiente.
- Realizzazione:
 - Paginazione su richiesta.
 - Segmentazione su richiesta.

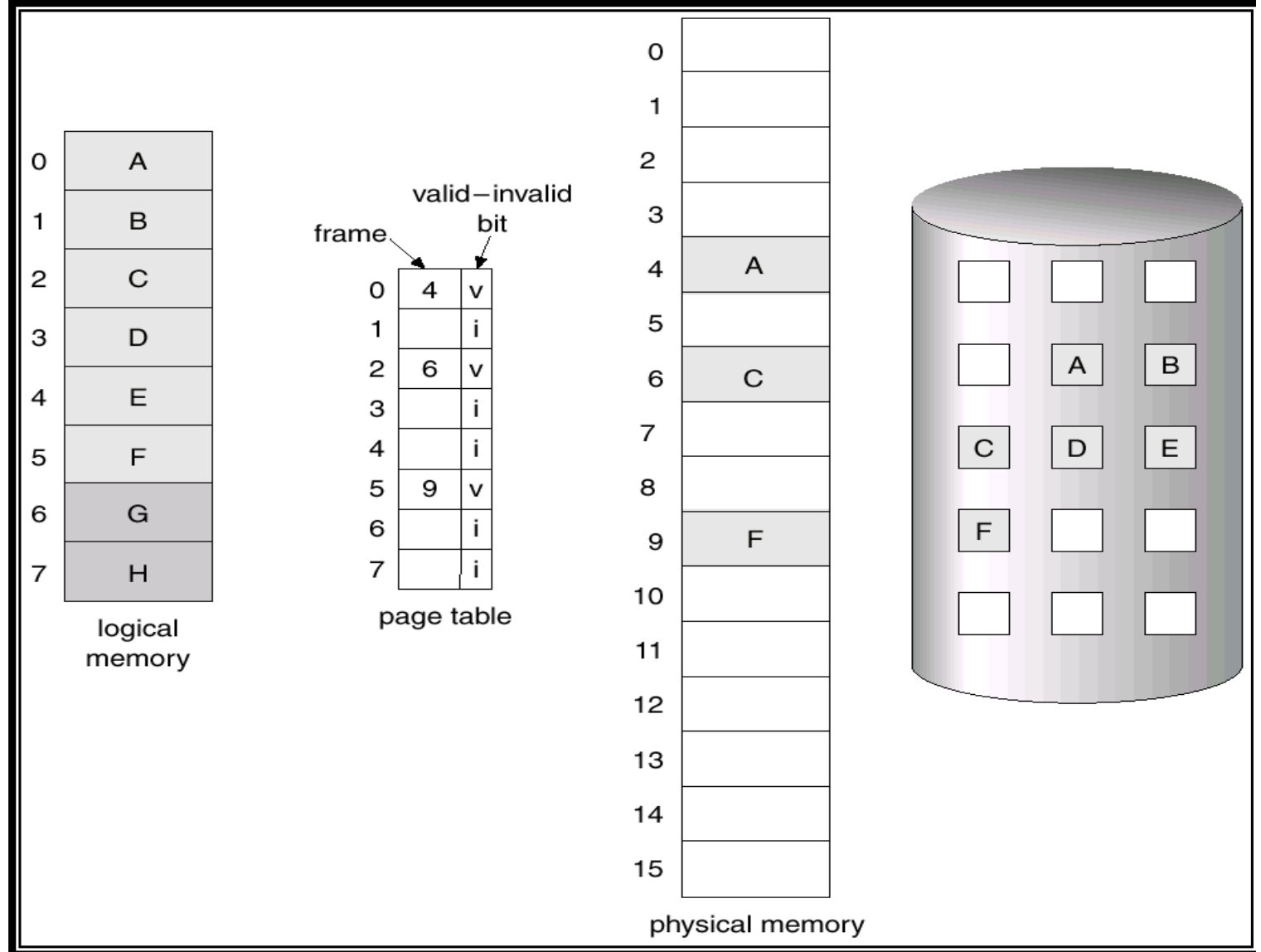


Paginazione su Richiesta

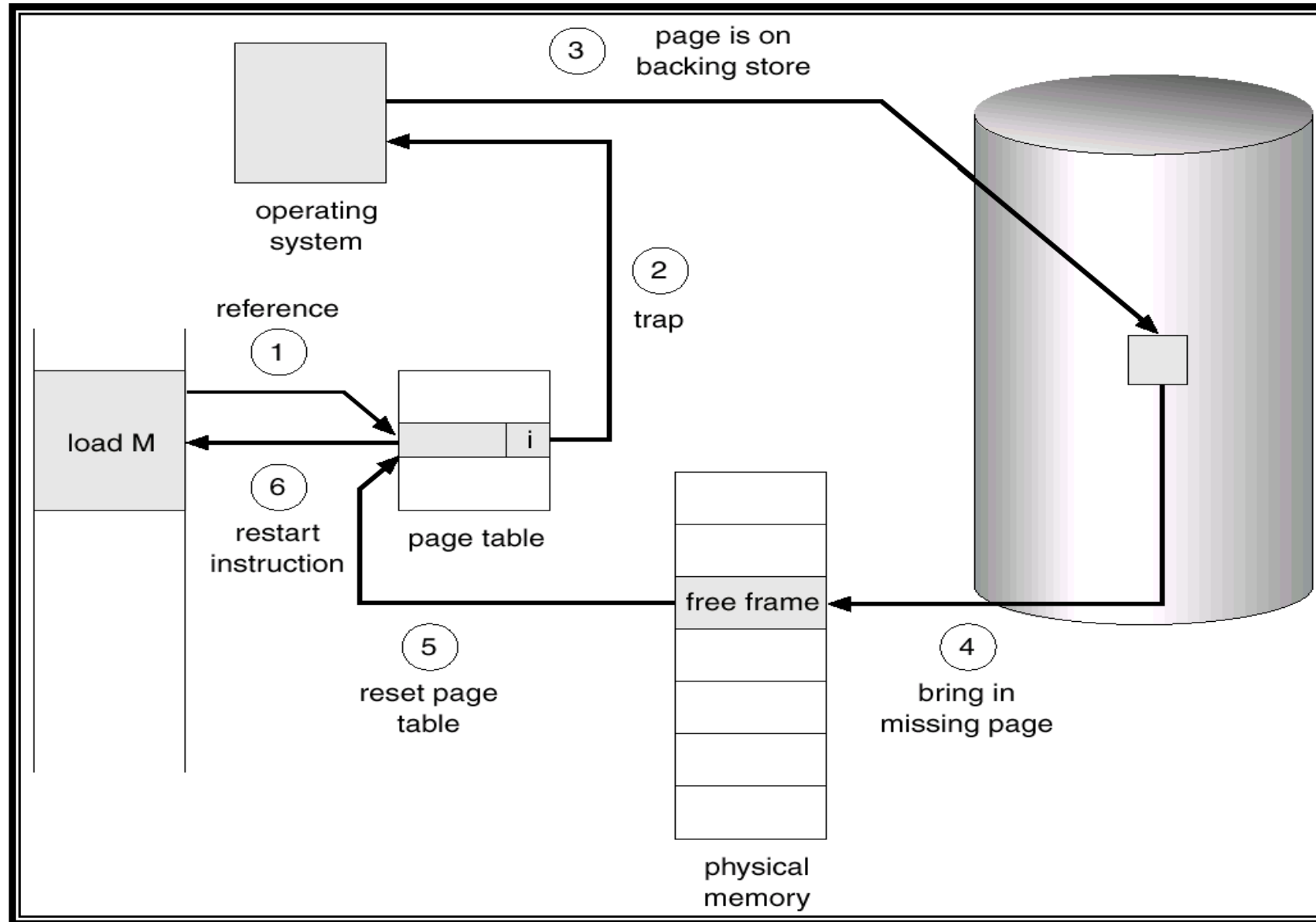


Paginazione su Richiesta

- Il bit di validità è inizializzato a 0.
- In fase di traduzione, se bit = 0 $\Rightarrow$ **page fault**.
 - Riferimento non valido
 - Pagina non in memoria



Gestione di un Page Fault



Prestazioni della Paginazione su Richiesta

- Probabilità che si verifichi un Page Fault: $0 \leq p \leq 1.0$
 - $p = 0$ page fault non si verifica mai
 - $p = 1$ page fault si verifica sempre
- **Tempo di accesso effettivo (EAT)**
$$\text{EAT} = (1 - p) \cdot \text{accesso_in_memoria} + p \cdot (\text{page_fault} + \text{swap_out} + \text{swap_in} + \text{restart})$$
- Esempio:
si desidera un rallentamento inferiore al 10% avendo le seguenti prestazioni:
 - $\text{accesso_in_memoria} = 0.1 \mu\text{s}$
 - $\text{page_fault} + \text{swap_out} + \text{swap_in} + \text{restart} = 25 \text{ ms} = 25000 \mu\text{s}$
$$\begin{aligned} \text{EAT} &= (1 - p) \cdot 0.1 + p \cdot 25000 = \\ &= 0.1 + 24999.9 \cdot p < 0.11 \end{aligned}$$
- dobbiamo avere

$$p < 4 \times 10^{-7}$$

- Swap-out

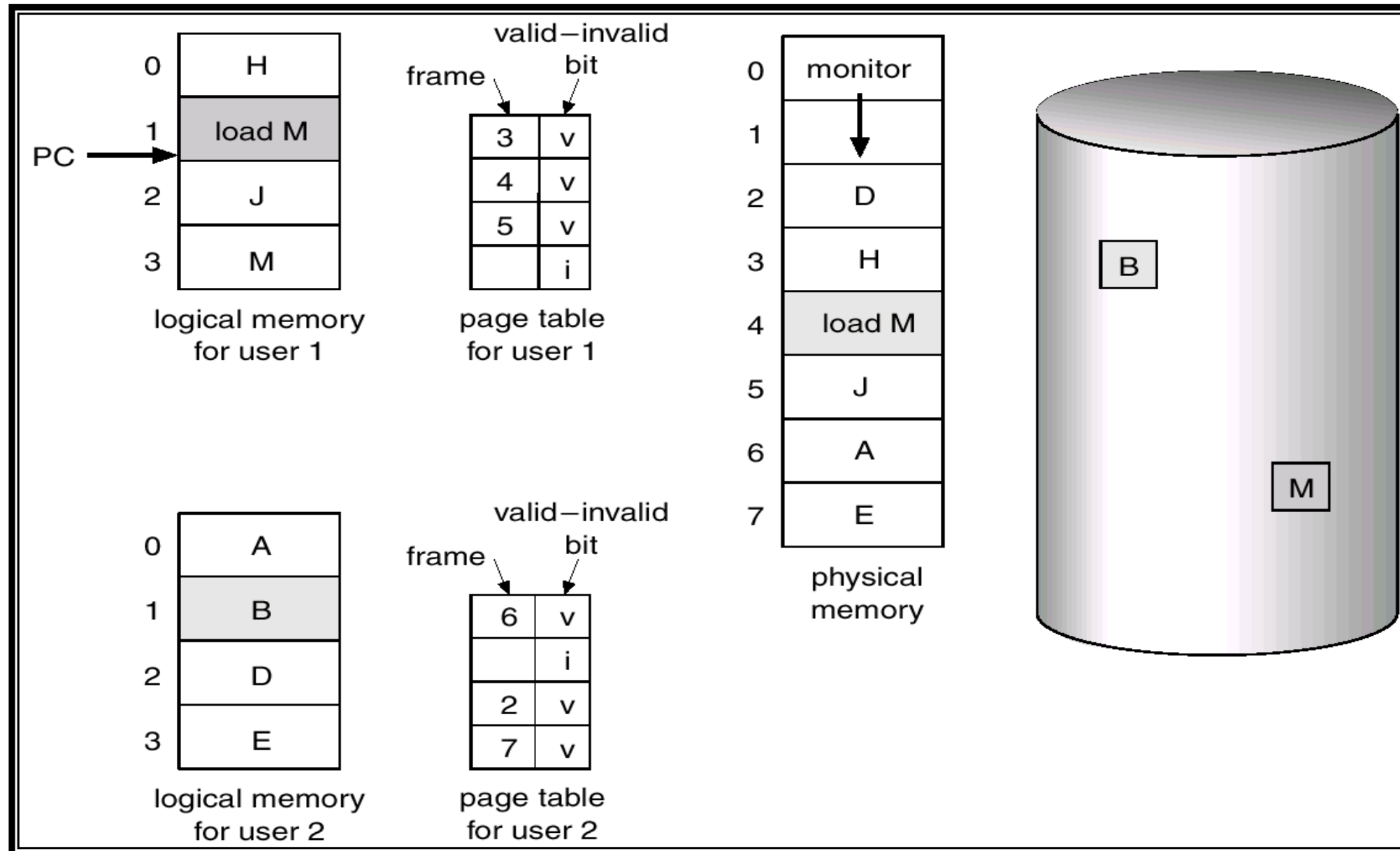
Per ridurre il numero di swap-out richiesti, si può utilizzare un **bit di modifica (dirty bit)**:

- Il bit viene messo ad 1 ogni volta che la pagina viene modificata
- Durante lo swap-out la pagina viene effettivamente copiata su disco solo se il bit di modifica è a 1.

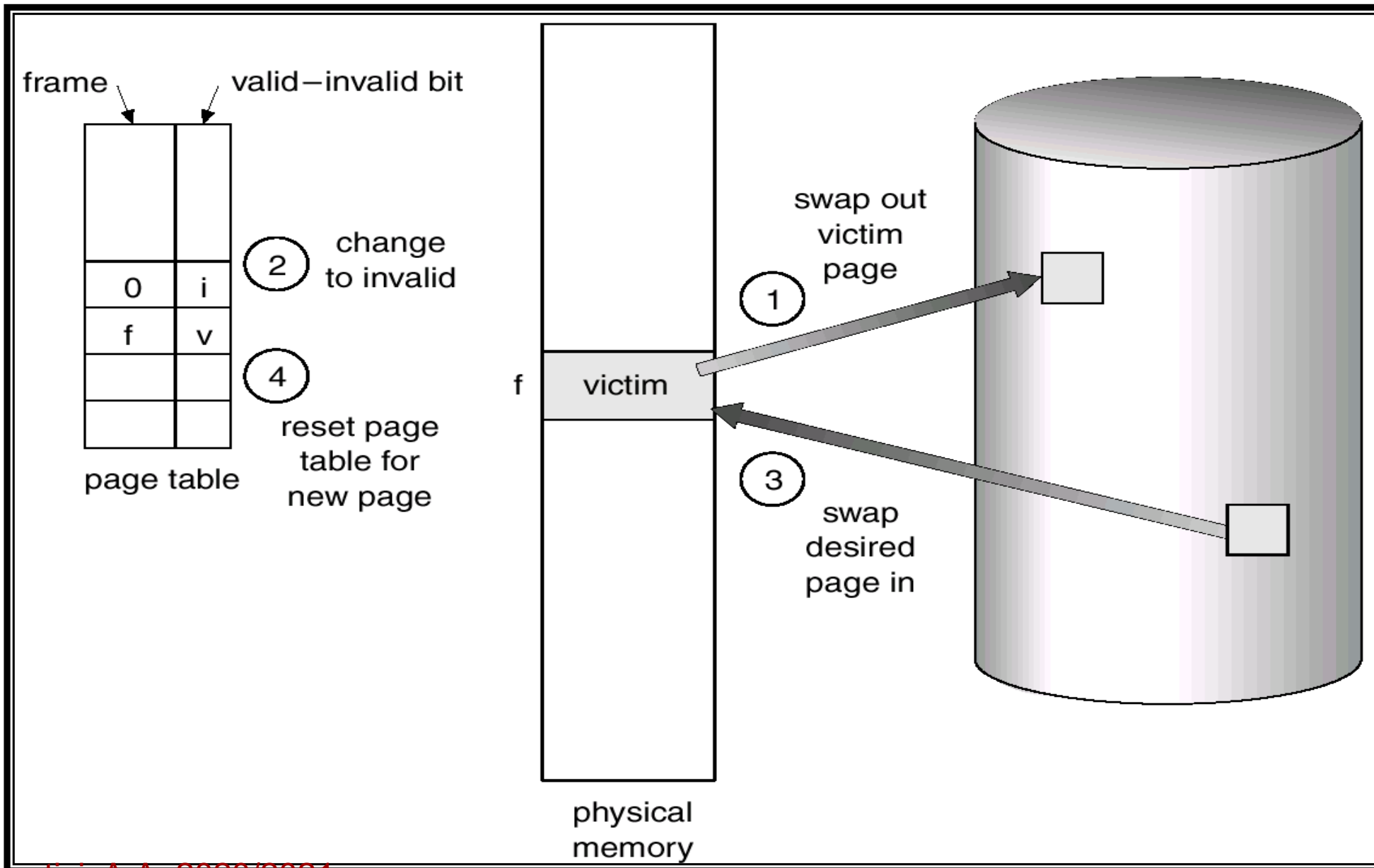
- Servono inoltre:

- **Algoritmo di assegnazione dei blocchi di memoria**
- **Algoritmo di sostituzione delle pagine.**

Sostituzione delle Pagine Necessità della Sostituzione



Sostituzione delle Pagine Algoritmo Base



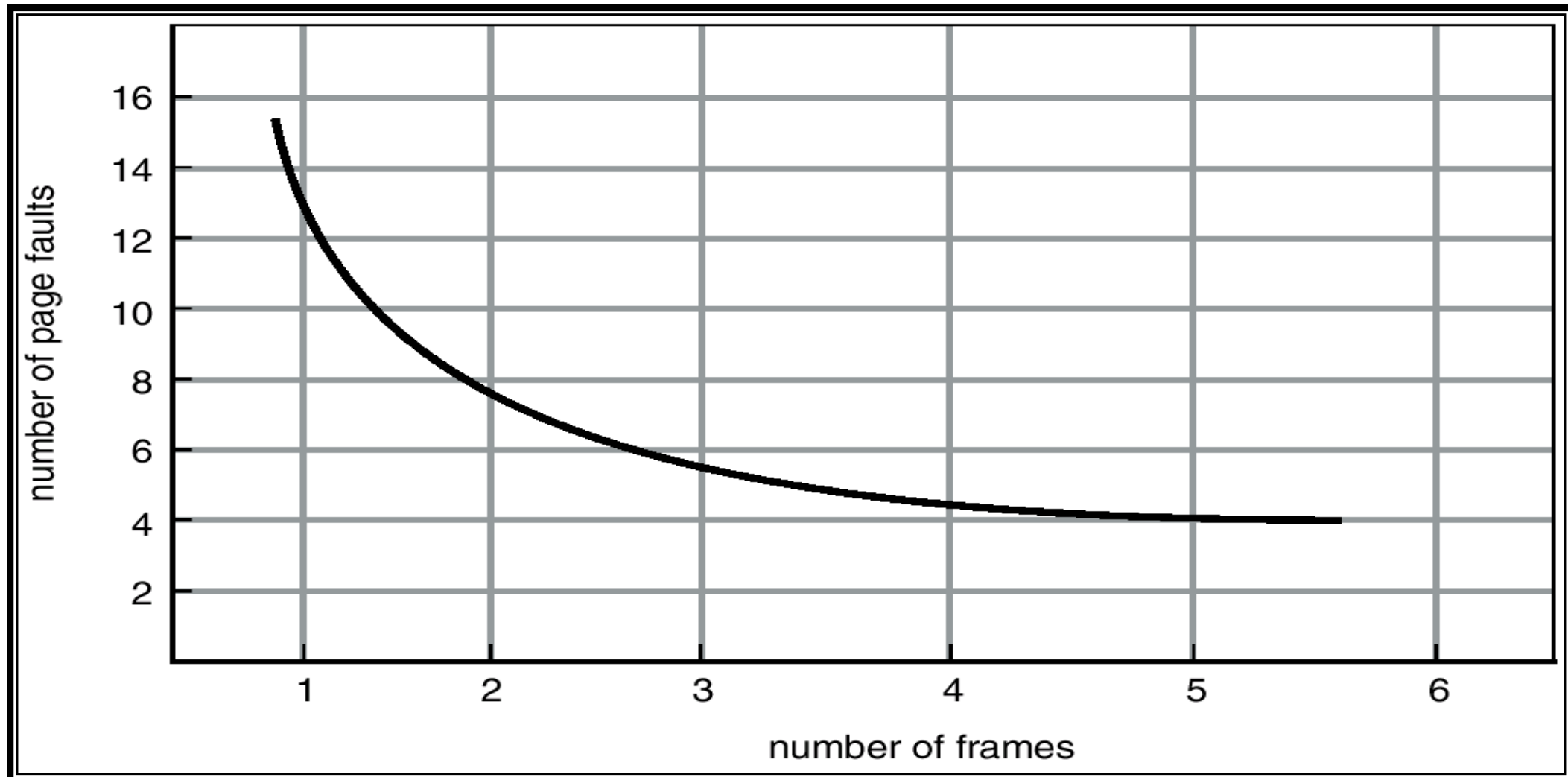


Sostituzione delle Pagine Criteri di Scelta dell' Algoritmo

- Deve minimizzare la probabilità di page-fault.
- Gli algoritmi vengono valutati considerando il loro comportamento (il numero di page fault prodotti) con determinate **successioni dei riferimenti**.
- In tutti i nostri esempi le successione dei riferimenti sono:
successione 1: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.
successione 2: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Sostituzione delle Pagine

Andamento Desiderato dei Page Fault in Funzione del Numero di Frame



Algoritmo First-In-First-Out (FIFO)

“Sostituire la prima pagina caricata in memoria”

- **Successione 1:** 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 frame (in ogni istante possono essere in memoria 3 pagine per processo)

1	1	4	5	
2	2	1	3	9 page fault
3	3	2	4	

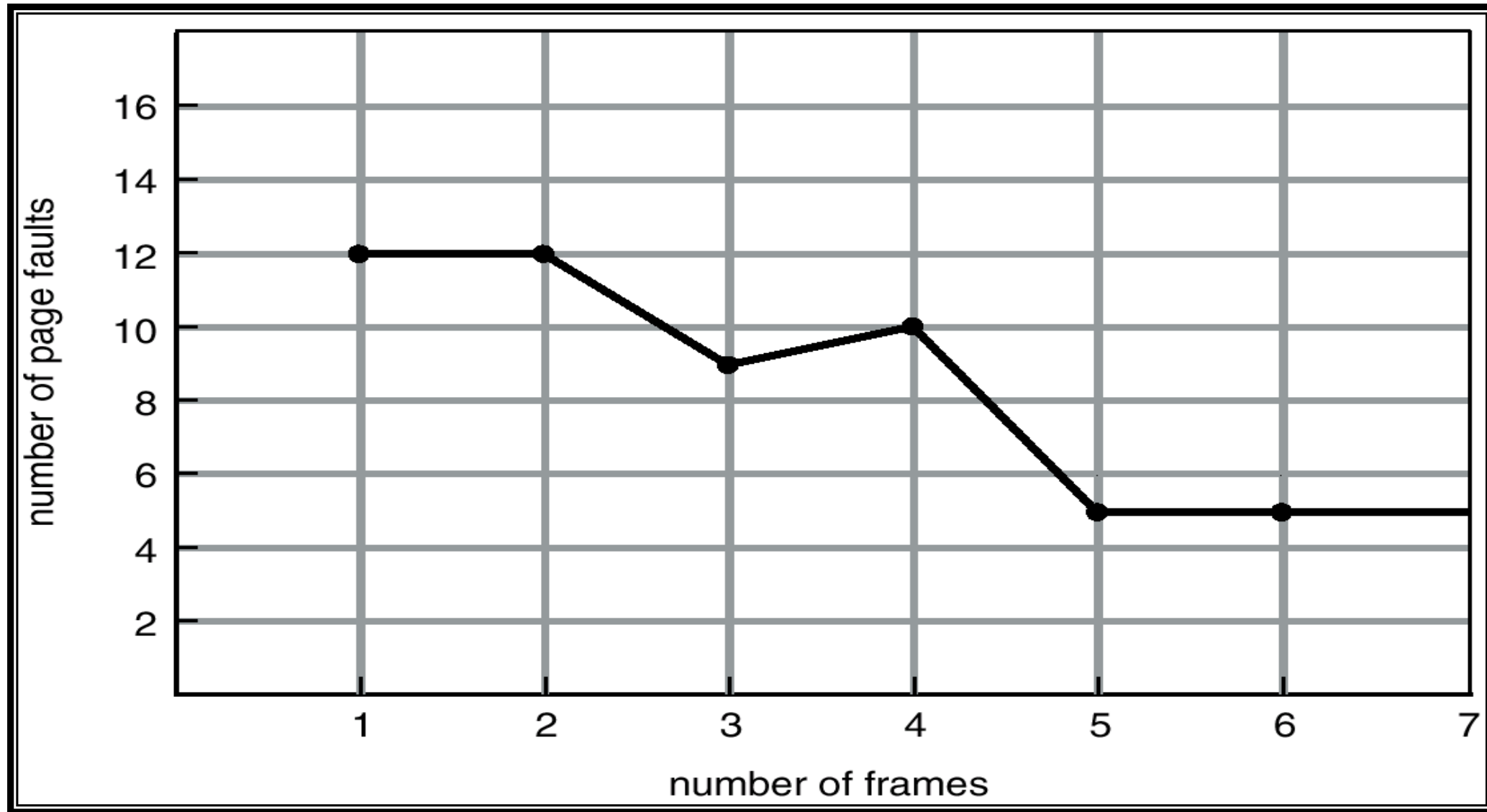
- 4 frame

1	1	5	4	
2	2	1	5	10 page fault
3	3	2		
4	4	3		

- **Anomalia di Belady**
 - Aumentare i frame non garantisce una diminuzione dei page fault

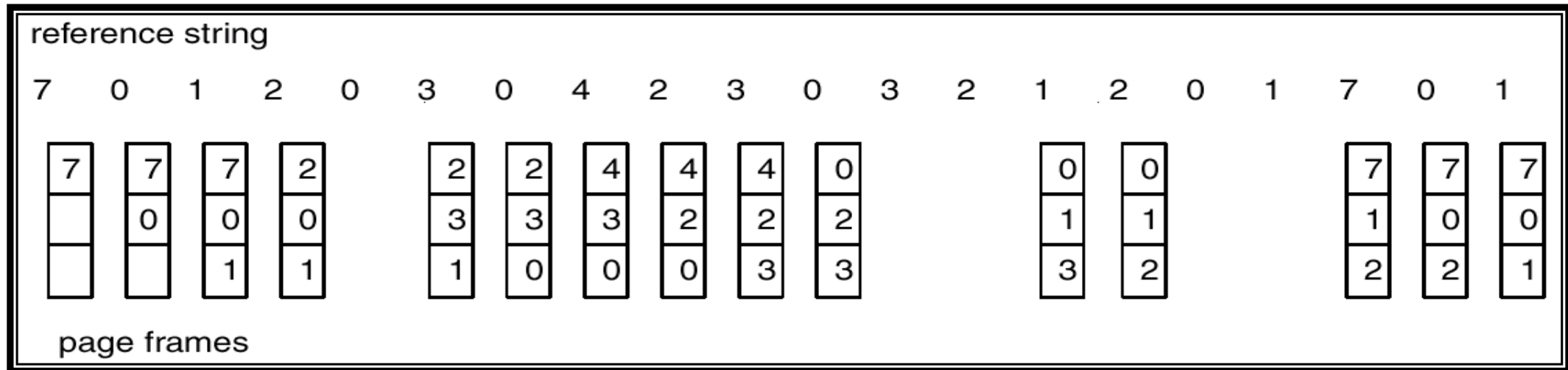


Algoritmo First-In-First-Out (FIFO) Anomalia di Belady



Algoritmo First-In-First-Out (FIFO)

Successione 2



“Sostituire la pagina che si utilizzerà il più tardi possibile”

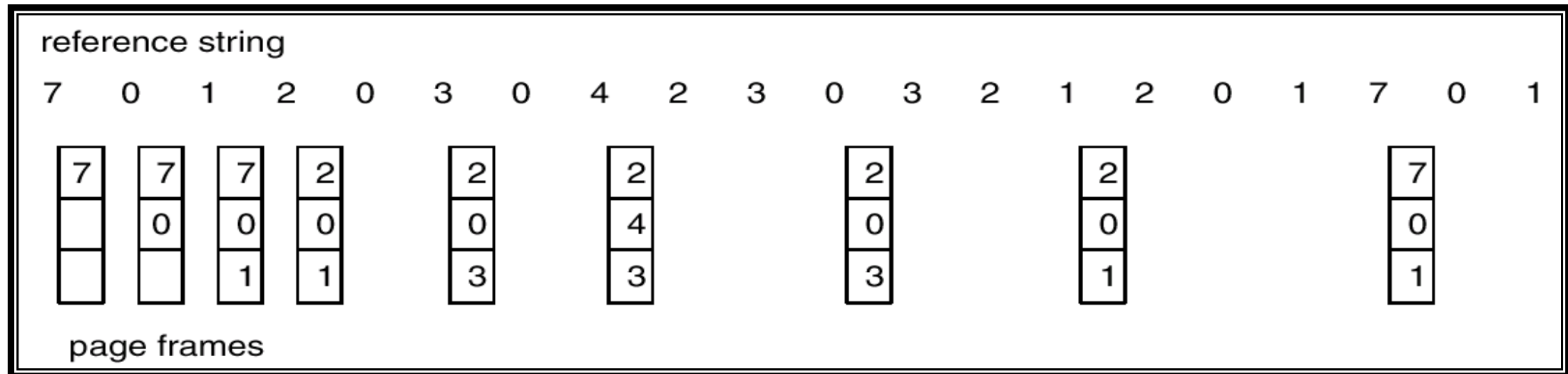
- **Successione 1:** 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 4 frame

1	4
2	
3	
4	5

6 page fault

- Impossibile da realizzare in pratica
- Usato come punto di riferimento per gli altri algoritmi.

Successione 2





Algoritmo Least Recently Used (LRU)

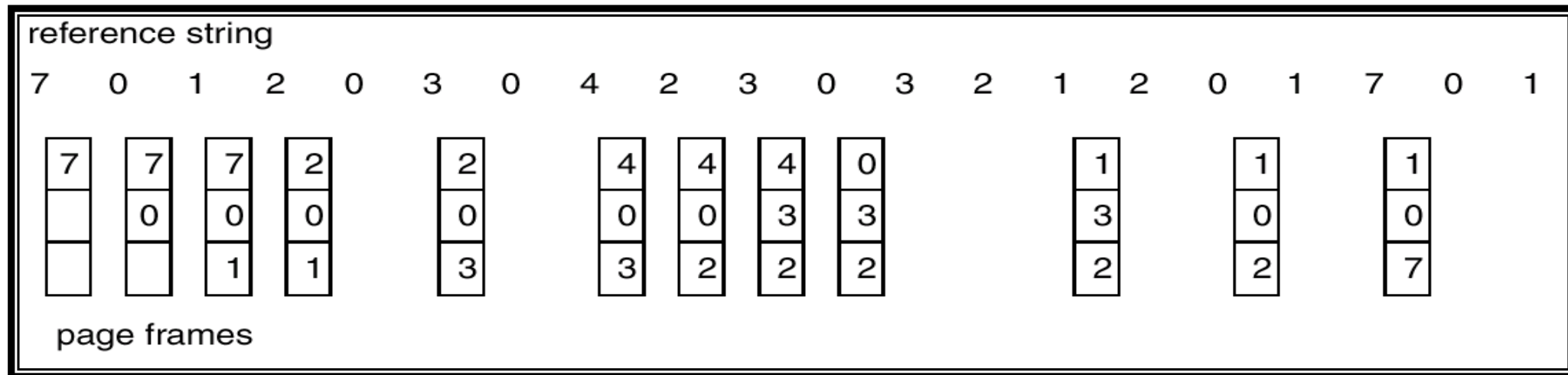
“Sostituire la pagina che è stata usata meno di recente”

- **Successione 1:** 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

1	5	
2		8 page fault
3	5	4
4	3	

Algoritmo Least Recently Used (LRU)

Successione 2





Algoritmo Least Frequently Used (LFU)

“Sostituire la pagina che è stata usata meno di frequente”

- **Successione 1:** 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

1		
2		
3	5	4
4	3	5

8 page fault

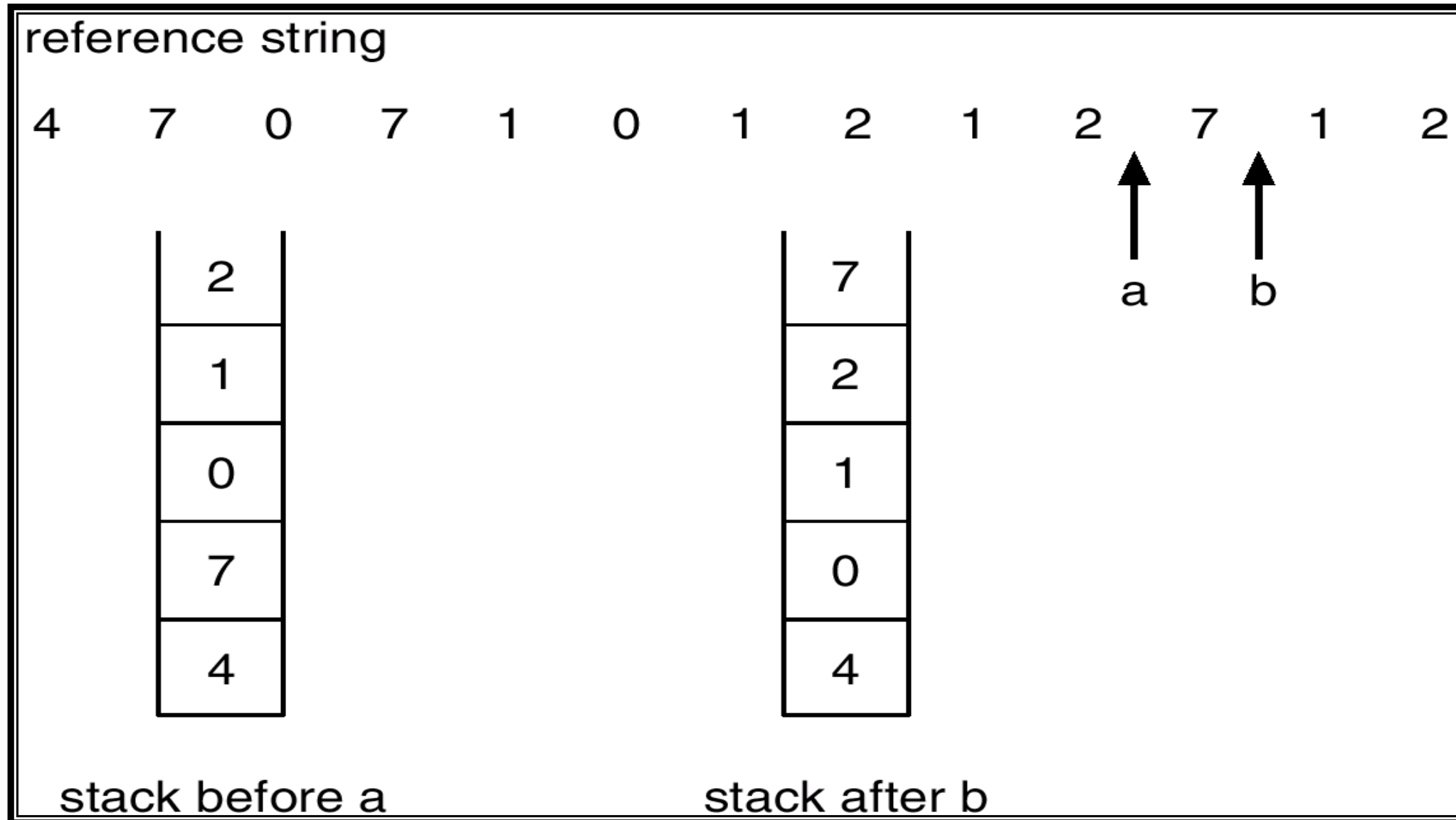
■ Con i **Contatori**

- Ogni elemento delle pagine ha un contatore.
- Ogni volta che c' è un riferimento alla pagina, il valore del clock viene copiato nel contatore $\Rightarrow$ **LRU**
- Ogni volta che c' è un riferimento alla pagina, il valore del contatore viene incrementato $\Rightarrow$ **LFU**
- Quando bisogna selezionare una pagina, si sceglie quella che ha il valore più piccolo del contatore.

■ Con lo **Stack**

- I numeri delle pagine sono organizzati in una lista doppiamente concatenata.
- Ogni volta che c' è un riferimento ad una pagina, il suo numero viene inserito/spostato in cima allo stack.
- Quando bisogna selezionare una pagina, si sceglie quella presente in fondo allo stack $\Rightarrow$ la sostituzione non richiede nessuna ricerca nella lista

Algoritmo Least Recently Used (LRU) Realizzazione con lo Stack



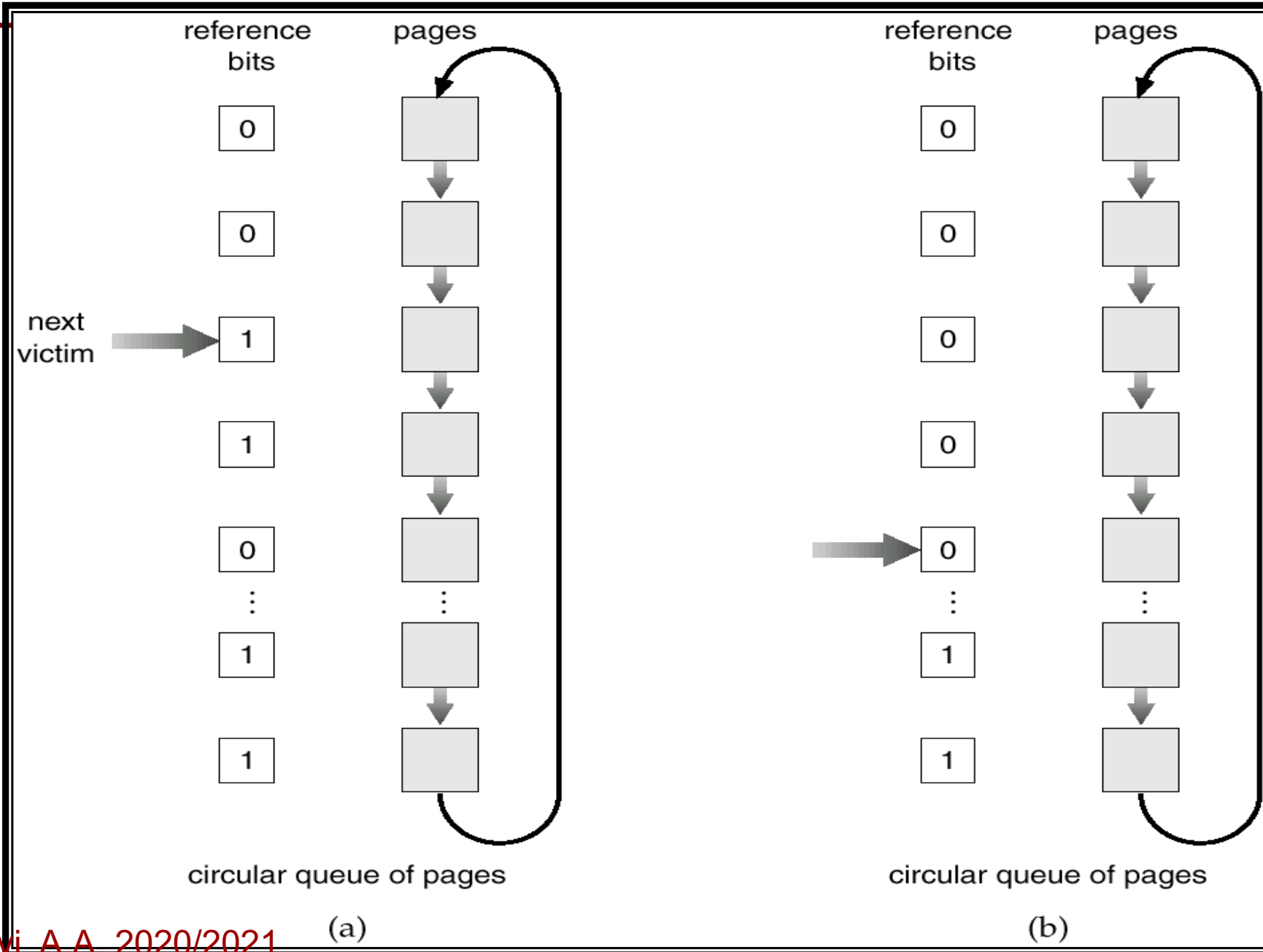
■ Bit di Riferimento

- Ad ogni pagina è associato un bit di riferimento, inizialmente = 0
- Ogni volta che c'è un accesso alla pagina il bit viene posto ad 1.
- La lista delle pagine è gestita FIFO, viene sostituita la pagina che ha il bit di riferimento a 0 (se esiste).
- Il bit dice se la pagina è stata usata recentemente, ma non mi dice l'ordine d'uso.

■ Seconda chance

- Utilizza il bit di riferimento.
- La lista delle pagine è una lista circolare (per questo viene detto anche algoritmo dell'orologio) gestita FIFO, viene selezionata una pagina:
- Se il bit di riferimento è a 0 => - viene sostituita
- Se il bit di riferimento è a 1 => - il bit viene messo a 0,
- la pagina è lasciata in memoria,
- viene selezionata la pagina successiva
a cui vengono applicate le stesse regole

Algoritmo della Seconda Chance (Orologio)





Esercizio: Algoritmo della Seconda Chance

- Applicare l' algoritmo della seconda chance alla seguente successione :

- **successione 2:**

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Assegnazione dei Blocchi di Memoria

- Nella paginazione su richiesta pura, inizialmente tutti i frame sono posti nella lista dei frame liberi, quando il primo processo deve essere caricato in memoria, il processo genera una sequenza di page fault.
- Un metodo più efficiente, consiste nell'assegnare al processo direttamente in fase di caricamento un certo numero di frame liberi.
- Ogni processo ha bisogno di un **numero minimo di pagine** che devono essere assegnate ad ogni processo. Tale numero dipende dal linguaggio macchina.
- Esempio: **IBM 370** – possono servire fino a 6 pagine per eseguire l'istruzione **SS MOVE <from> <to>**:
 - L'istruzione occupa 6 byte, quindi può stare a cavallo di due pagine.
 - Il contenuto della parola indirizzata da <from> può stare a cavallo di due pagine.
 - Il contenuto della parola indirizzata da <to> può stare a cavallo di due pagine.



Algoritmo di Assegnazione Uniforme

- I frame vengono assegnati in base al numero di processi:
sia m il numero di frame liberi ed n il numero di processi,
ogni processo riceve m/n frame.
- **Esempio**
 - $m = 100$ frame
 - $n = 5$ processi,
 - ogni processo riceve 20 pagine.

Algoritmo di Assegnazione Proporzionale

- Con l'assegnazione uniforme ci sono processi di piccole dimensioni che ricevono molti frame e processi di grandi dimensioni che ricevono pochi frame.
- Con l'assegnazione proporzionale i frame vengono assegnati in base alle dimensioni del processo:
 - s_i dimensioni del processo p_i ,
 - $S = \sum s_i$
 - m = numero totale di frame
 - $a_i = m \times s_i / S$ numero di frame assegnati al processo p_i

- **Esempio**

$m = 62$ $s_1 = 10$ $s_2 = 127$

Uniforme

$$a_1 = 62/2 = 31$$

$$a_2 = 62/2 = 31$$

Proporzionale

$$a_1 = 62 \times 10/137 \approx 4$$

$$a_2 = 62 \times 127/137 \approx 57$$

rimane 1 frame libero

Algoritmo di Assegnazione per Priorità

- Sia con l'assegnazione uniforme sia con l'assegnazione proporzionale, un processo con priorità elevata viene trattato nello stesso modo di un processo a bassa priorità.
- L'assegnazione per priorità si basa su una assegnazione proporzionale alla priorità dei processi invece che alla loro dimensione.
- Se un processo P_i genera un page fault,
 - Si sostituisce uno dei suoi frame (**assegnazione locale**).
 - Si sostituisce un frame di un processo con priorità minore (**assegnazione globale**).Questo permette ad un processo con una priorità elevata di aumentare il numero di frame che gli sono stati assegnati.



Assegnazione Globale e Locale

- **Assegnazione Globale** –

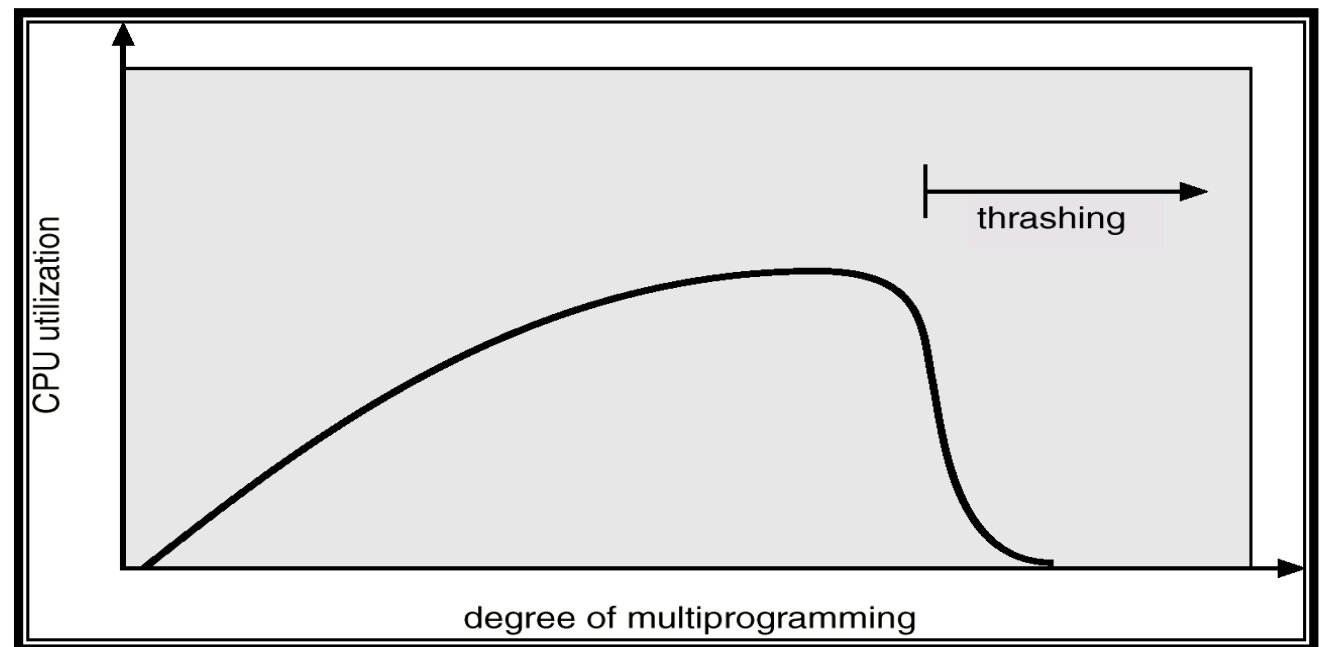
per la sostituzione viene selezionato un frame tra tutti quelli esistenti, anche se appartiene ad un processo diverso da quello che ha generato il page fault. In questo modo ci sono processi che vedono aumentare il numero di frame a loro disposizione e processi che vedono tale numero diminuire.

- **Assegnazione Locale** –

per la sostituzione viene selezionato un frame del processo che ha generato il page fault.

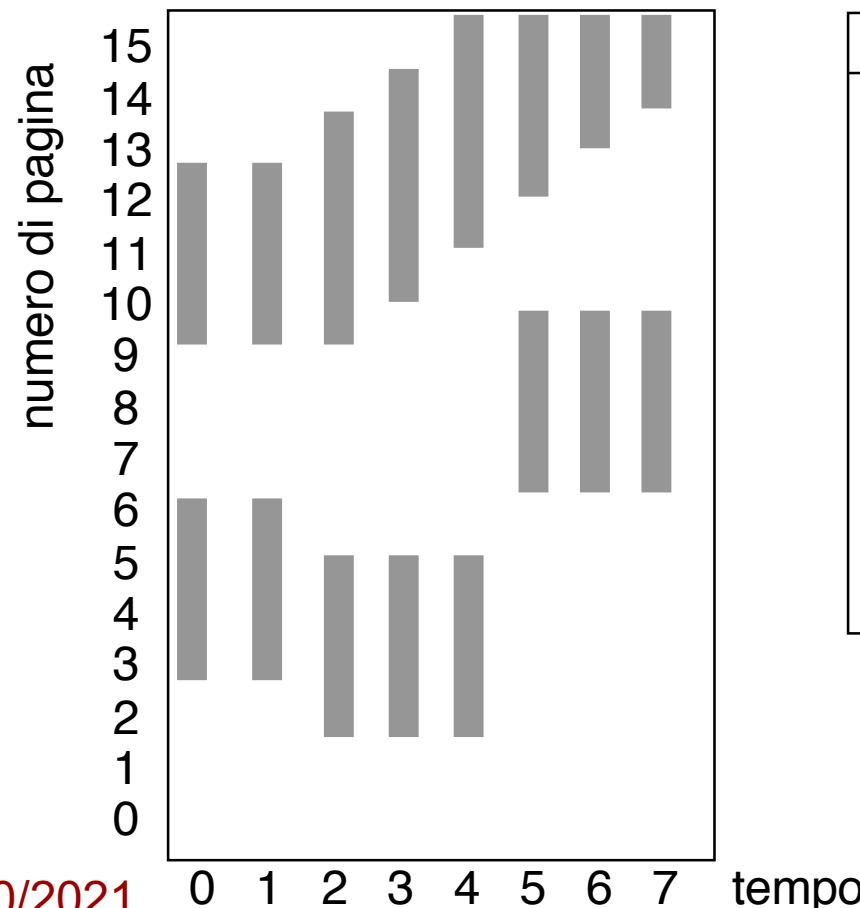
Paginazione Degenere (Thrashing)

- Se un processo non ha un numero “sufficiente” di pagine, il tasso di page fault diventa troppo alto. Questa può essere la conseguenza della seguente situazione
 - se si verifica un basso utilizzo della CPU
 - il SO decide di aumentare il grado di multiprogrammazione
 - aggiunge un altro processo in memoria.
- **Thrashing** $\equiv$ *il SO impiega gran parte del tempo in operazioni di swap-in e swap-out.*
- Questo si verifica soprattutto se si usa un algoritmo di assegnazione globale.



Principio di Località

- Le pagine usate da in processo in un certo periodo della sua esistenza formano la località in cui si trova il processo in quel momento.
- Un processo migra da una località ad un'altra.
- Le località si possono sovrapporre tra loro.
- Il trashing si verifica se: $\sum \text{dim(località)} > \text{dim(memoria)}$

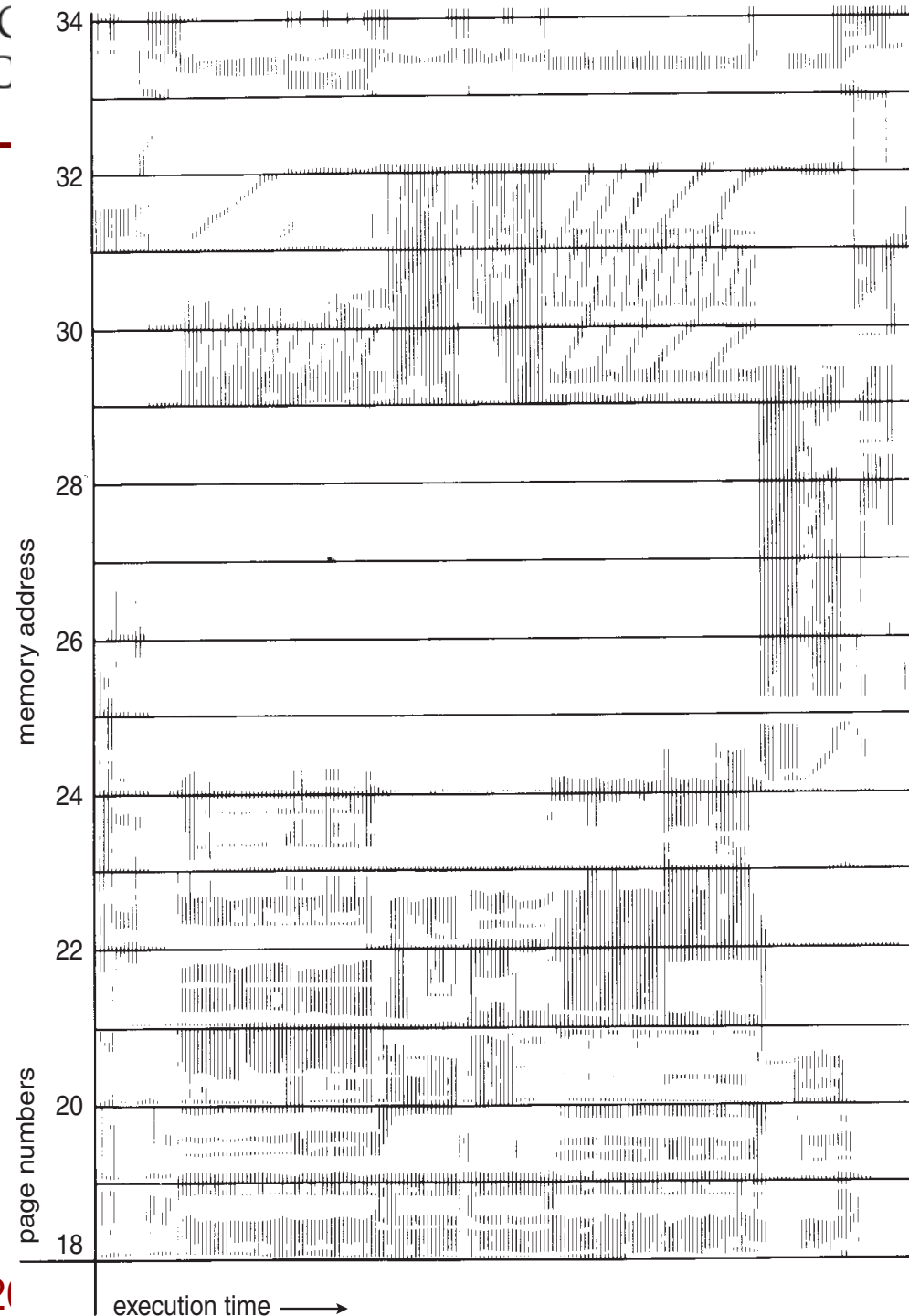


tempo	pagine usate
0	2, 3, 4, 5, 9, 10, 11, 12
1	2, 3, 4, 5, 9, 10, 11, 12
2	1, 2, 3, 4, 9, 10, 11, 12, 13
3	1, 2, 3, 4, 10, 11, 12, 13, 14
4	1, 2, 3, 4, 11, 12, 13, 14, 15
5	6, 7, 8, 12, 13, 14, 15
6	6, 7, 8, 13, 14, 15
7	6, 7, 8, 14, 15

Pagine usate da un processo P
nell'intervallo di tempo [0, 7]

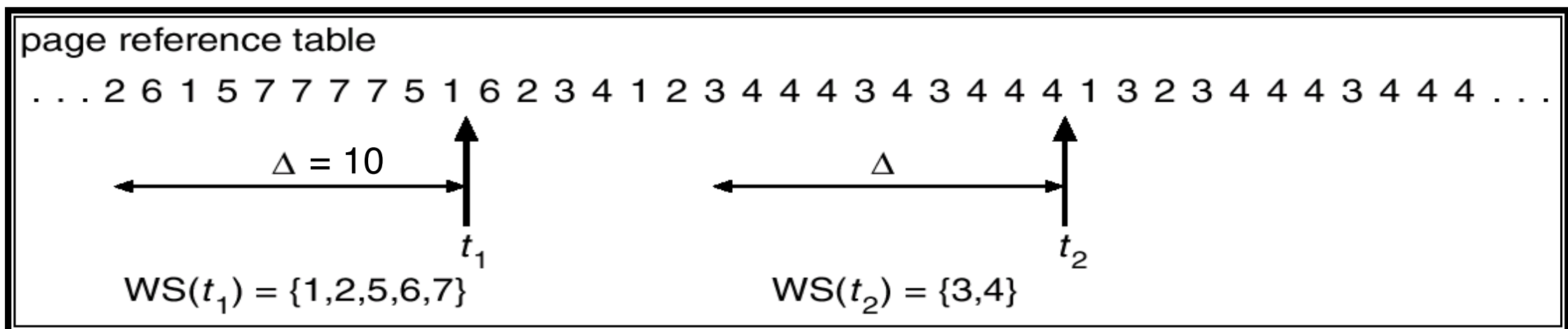


Principio di Località



Assegnazione Basata sul Working-Set

- Working-set di un processo $\equiv$ insieme delle pagine referenziate negli ultimi Δ istanti (dove $\Delta \equiv$ finestra del working-set).
- WSS_i = dimensione del working-set del processo P_i (varia nel tempo)
 - se Δ troppo piccolo non include l'intera località.
 - se Δ troppo grande può sovrapporre più località.
 - se $\Delta = \infty \Rightarrow$ include l'intero processo.
- $D = \sum WSS_i \equiv$ totale della domanda di frame (se $D > m \Rightarrow$ Thrashing)
- Schema basato sul working-set
 - assegnazione pari a dimensione del working set (varia nel tempo)
 - se $D < m$, allora viene attivato un nuovo processo;
 - se $D > m$, allora un processo viene sospeso.





Esercizio: Working Set

Calcolare l'evoluzione temporale del working set nel seguente caso:

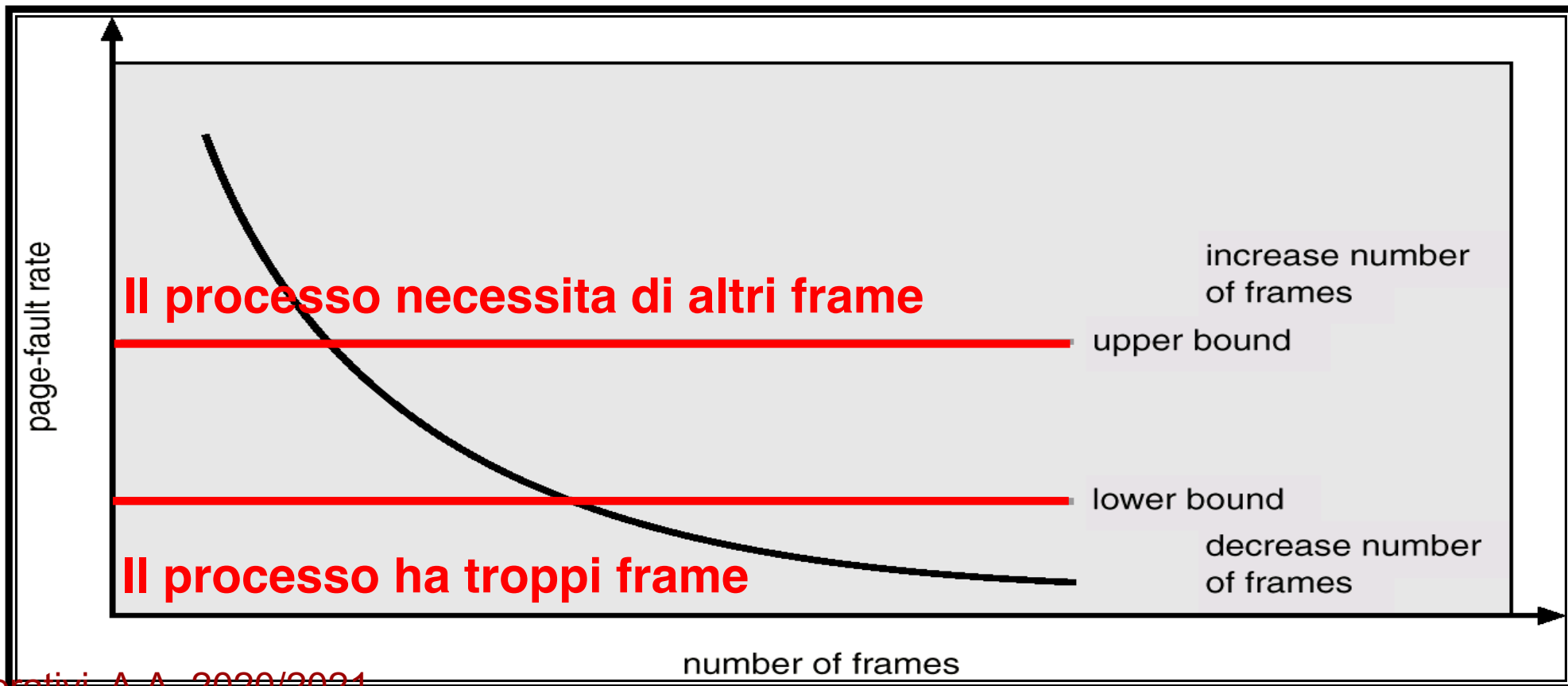
- ampiezza finestra: $\Delta = 10$
- frequenza: ogni 5 riferimenti

- **successione 2:**

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

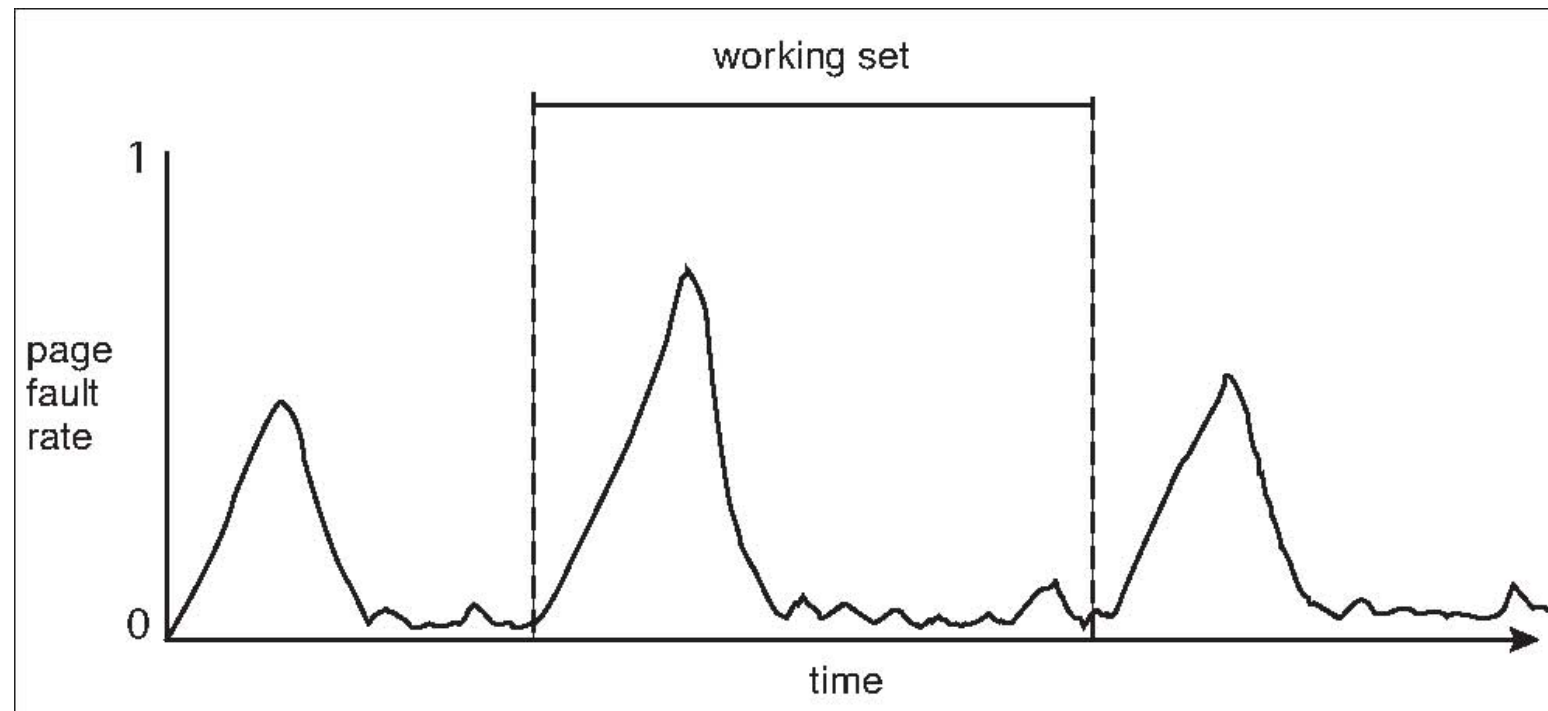
Assegnazione Basata su Page Fault Rate

- Schema basato sulla frequenza di page-fault.
 - Se la frequenza scende sotto il limite inferiore, al processo viene tolto un frame.
 - Se la frequenza sale sopra il limite superiore, al processo viene assegnato un altro frame.



Working Set e Page Fault Rate

- Relazione diretta tra il working set di un processo e il suo page fault rate
 - Il working set cambia nel tempo
 - anche i picchi e le valli cambiano nel tempo



■ Esempio

- Supponiamo di avere pagine di dimensionipari a 1024 parole.
- Supponiamo di dover inizializzare a zero tutti gli elementi della seguente matrice

```
int A[1024][1024];
```

- Ogni riga è memorizzata in una pagina.

— Programma 1

```
for (j = 0; j < 1024; j++)  
    for (i = 0; i < 1024; i++)  
        A[i,j] = 0;
```

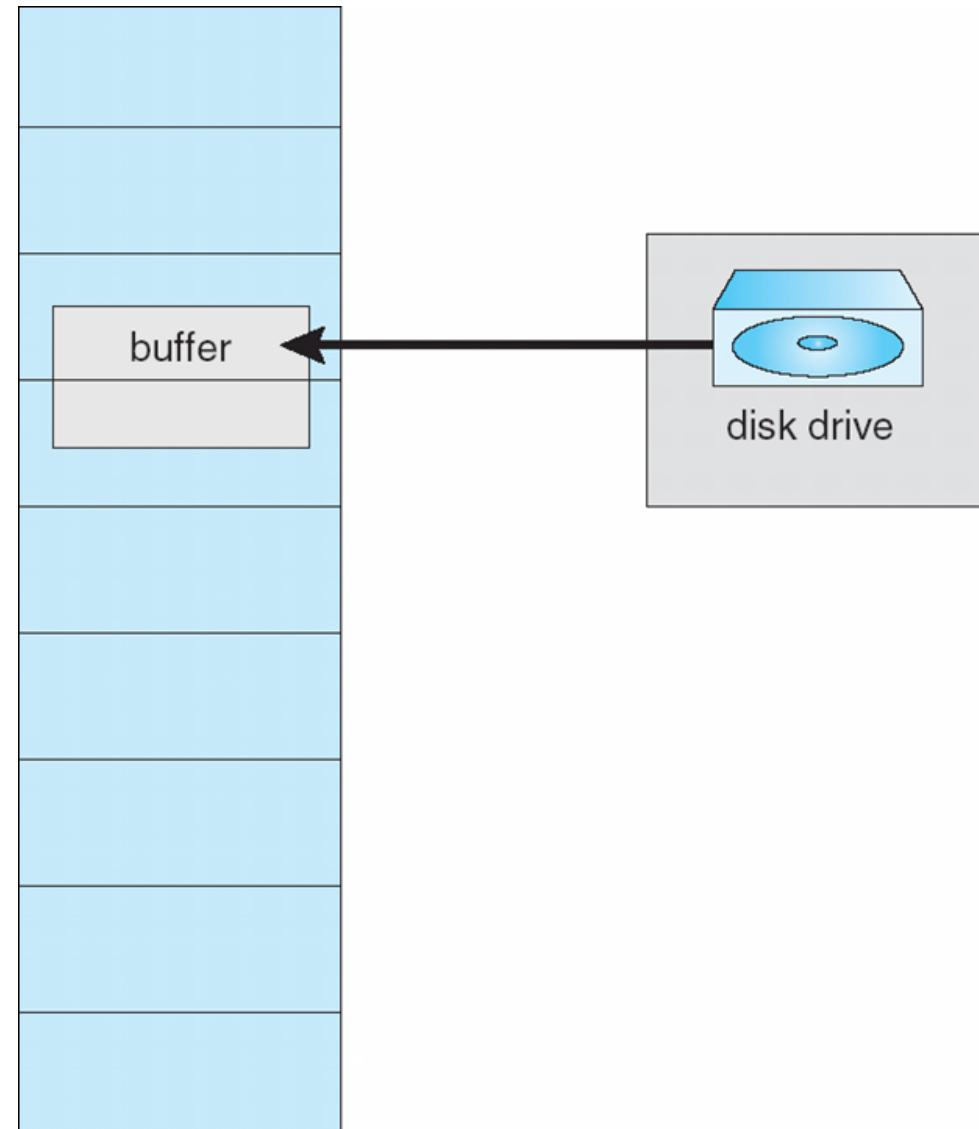
1024 x 1024 page fault

— Programma 2

```
for (i = 0; i < 1024; i++)  
    for (j = 0; j < 1024; j++)  
        A[i,j] = 0;
```

1024 page fault

- Le pagine che sono usate per copiare un file da un dispositivo di I/O devono poter essere vincolate in memoria per evitare che vengano selezionate dall'algoritmo di sostituzione delle pagine.





Memoria

GESTIONE MEMORIA MULTIPROCESSORI

- ✓ Introduzione
 - ✓ Codice rilocabile
 - ✓ Swapping
- ✓ Gestione della Memoria
 - ✓ Allocazione contigua
 - ✓ Paginazione
 - ✓ Segmentazione
- ✓ Memoria Virtuale
 - ✓ Paginazione su richiesta
 - ✓ Algoritmi di sostituzione delle pagine: FIFO, Ottimale, LRU, Orologio
 - ✓ Algoritmi di assegnazione dei frame liberi: Uniforme, Proporzionale, per Priorità, basata sul Working Set, basata su Page Fault Rate
- Gestione Memoria Multiprocessori
- Gestione della Memoria in Linux
 - Memoria Fisica
 - Memoria Virtuale



- Tutti i processori condividono lo stesso spazio di indirizzi fisici con la stessa latenza
- Semplice estensione della memoria virtuale per sistemi monoprocessore

Multiprocessori: Sistemi NUMA

- Ogni processore ha una sua memoria locale ma può accedere anche alla memoria degli altri processori
 - È preferibile accedere alla propria memoria locale che ha un tempo di accesso minore
- Se la pagina non si trova nella memoria locale ma nella memoria di un altro processore?
 - Tre possibili soluzioni:
 - Migrazione
 - Replicazione
 - Nessuna delle due
 - Strategia di scelta
 - Molti sistemi mantengono la storia di ogni accesso alle pagine (frequenza accessi, da quale processore, tipo di accesso, ...) per decidere se adottare la replicazione, la migrazione, o nessuna delle due



Memoria

GESTIONE DELLA MEMORIA LINUX

- ✓ Introduzione
 - ✓ Codice rilocabile
 - ✓ Swapping
- ✓ Gestione della Memoria
 - ✓ Allocazione contigua
 - ✓ Paginazione
 - ✓ Segmentazione
- ✓ Memoria Virtuale
 - ✓ Paginazione su richiesta
 - ✓ Algoritmi di sostituzione delle pagine: FIFO, Ottimale, LRU, Orologio
 - ✓ Algoritmi di assegnazione dei frame liberi: Uniforme, Proporzionale, per Priorità, basata sul Working Set, basata su Page Fault Rate
- ✓ Gestione Memoria Multiprocessori
 - Gestione della Memoria in Linux
 - Memoria Fisica
 - Memoria Virtuale

Gestione della Memoria in Linux

Spazio di indirizzamento di un task per **Linux a 32 bit**

- Abbiamo al massimo **4GB** di memoria.
- Ogni task ha a disposizione uno spazio d indirizzamento virtuale di 3GB, il restante 1GB è riservato al kernel.
- Un task in user mode non vede lo spazio riservato al kernel, riesce a vederlo nel momento in cui passa in kernel mode.
- Lo spazio di indirizzamento è costituito da un insieme di regioni omogenee e contigue.
- Ogni regione è costituita da un insieme di pagine con stesse proprietà di protezione e di paginazione.
- Ogni pagina ha dimensione pari a **4KB** su architetture x86

Gestione della Memoria in Linux

Spazio di indirizzamento di un task

Linux crea un novo spazio di indirizzamento virtuale quando:

- Viene eseguita una **exec**
 - Viene creato uno spazio di indirizzi completamente vuoto.
 - È compito del caricatore popolare lo spazio con regioni di memoria virtuale.
- Viene eseguita una **fork**
 - Viene creata una copia dello spazio di indirizzamento del task genitore.
 - Il kernel copia tutti i `vm_area_struct` del genitore.
 - Crea un nuovo insieme di tabelle delle pagine per il figlio.
 - Il contenuto delle tabelle delle pagine del genitore è copiato nelle tabelle del figlio.
 - Dopo la fork, genitore e figlio condividono le stesse pagine fisiche (anche quelle private del genitore).
 - La pagina di una regione privata viene duplicata solo in fase di esecuzione, quando uno dei due task chiede di modificarla.
 - Così le copie delle pagine si creano solo quando è necessario.

Gestione della Memoria in Linux

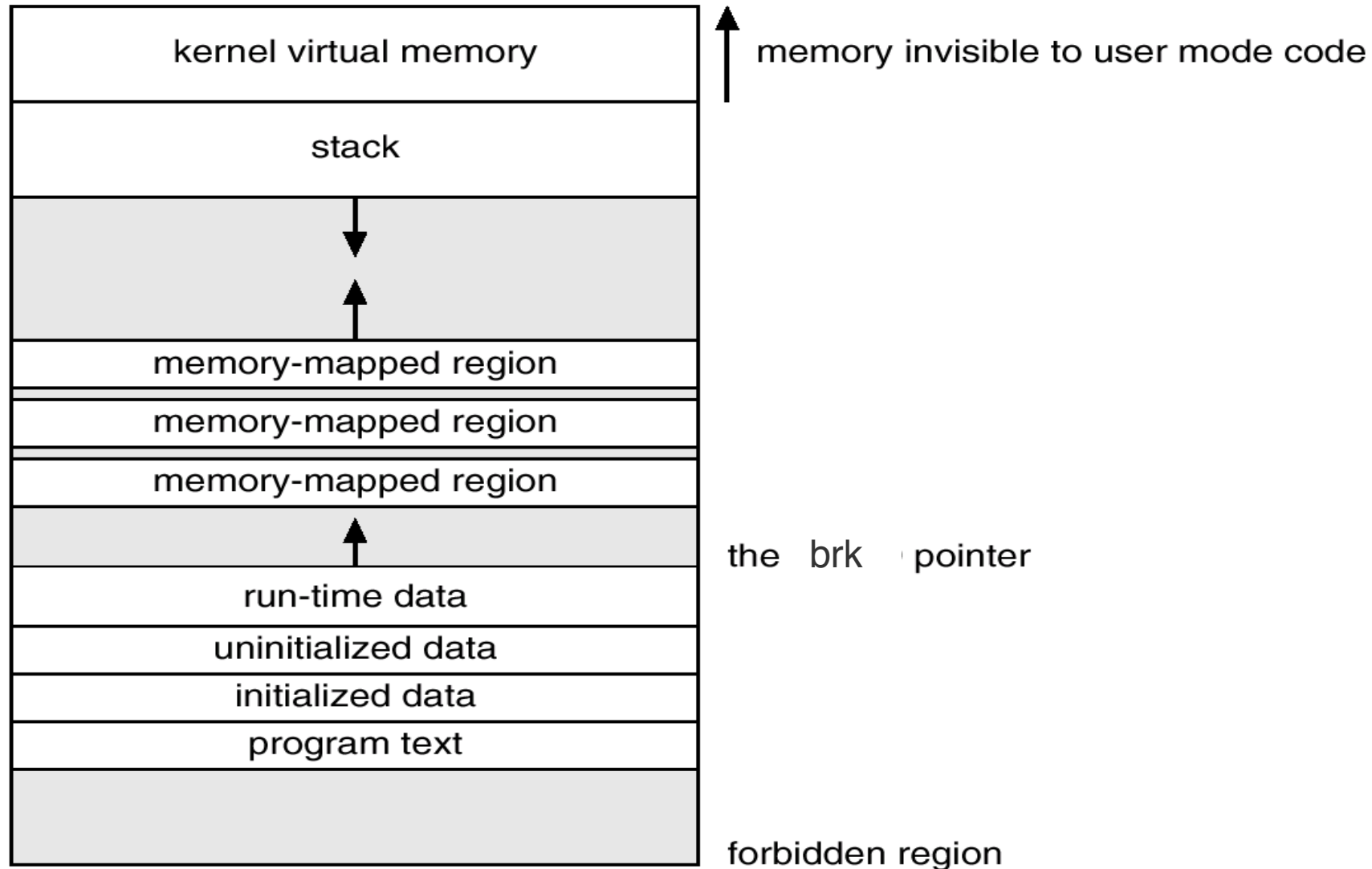
Spazio di indirizzamento di un task

- Linux adotta per i programmi eseguibili:
 - Il formato a.out (comune ai vecchi sistemi Unix);
 - Il formato ELF.
- Inizialmente il programma eseguibile viene mappato nello spazio di indirizzamento virtuale, ma le pagine non vengono caricate
 - solo quando il task chiede di accedere ad una data pagina, ci sarà un page fault e di conseguenza la pagina verrà caricata (swapped in) nella memoria fisica.
 - Il compito del caricatore è quello di instaurare le associazioni iniziali per permettere l'avvio dell'esecuzione del programma.
- Linux permette due tipi di collegamento alle librerie di sistema:
 - Il collegamento statico
 - Il collegamento dinamico (occupa meno spazio sia in memoria centrale sia su disco)

Gestione della Memoria in Linux

Spazio di indirizzamento di un task

- Organizzazione della memoria per il formato ELF

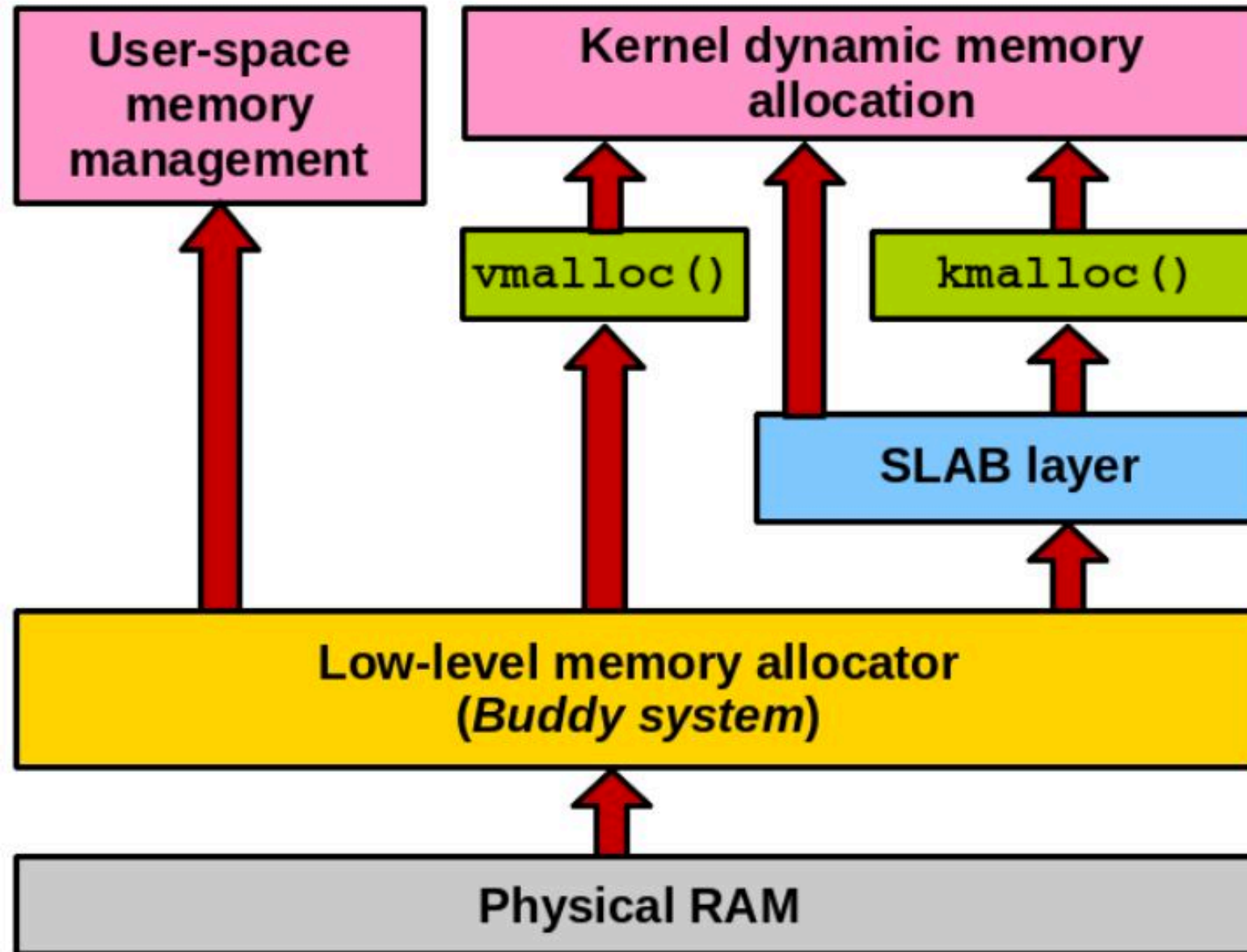


Gestione della Memoria in Linux

Il gestore della memoria di Linux ha due tipi di gestori:

- **Gestore della memoria fisica.** Si occupa dell'assegnazione e del rilascio di pagine fisiche, gruppi di pagine fisiche e piccoli blocchi di memoria (blocchi di memoria di dimensione inferiore alla pagina).
- **Gestore della memoria virtuale.** Si occupa della gestione della memoria virtuale, ovvero la memoria indirizzabile dai task in esecuzione.

Gestione della Memoria in Linux



- L'allocazione della memoria può avvenire in due modi:
 - staticamente – i driver riservano aree di memoria contigua durante l'avvio del sistema;
 - dinamicamente – tramite un allocatore delle pagine (quello base oppure per alcuni moduli del kernel dei propri allocatori specializzati)
- Allocatore delle pagine di base (**zoned buddy allocator**):
 - è responsabile dell'allocazione e del rilascio di pagine fisiche,
 - è in grado di assegnare su richiesta gruppi di pagine fisicamente contigue (regioni),
 - utilizza l'algoritmo buddy-heap.
- Allocatore delle pagine del kernel (**slab allocator**)
 - è responsabile dell'allocazione delle pagine del kernel in modo da ottimizzare l'uso della cache
 - utilizza l'algoritmo SLAB

Algoritmo **buddy-heap**:

- Ogni regione allocabile ha:
 - dimensione pari a 2^n pagine fisiche (quindi la regione più piccola ha dimensione pari ad una pagina),
 - una compagna adiacente (detta buddy).
- Per gestire le regioni libere, vengono utilizzate liste concatenate distinte, una per ogni dimensione.
- Ad ogni rilascio di memoria:
 - Se la regione rilasciata è adiacente ad un'altra regione libera, le due regioni vengono combinate per formarne una più ampia.

Algoritmo **buddy-heap** (*continua*):

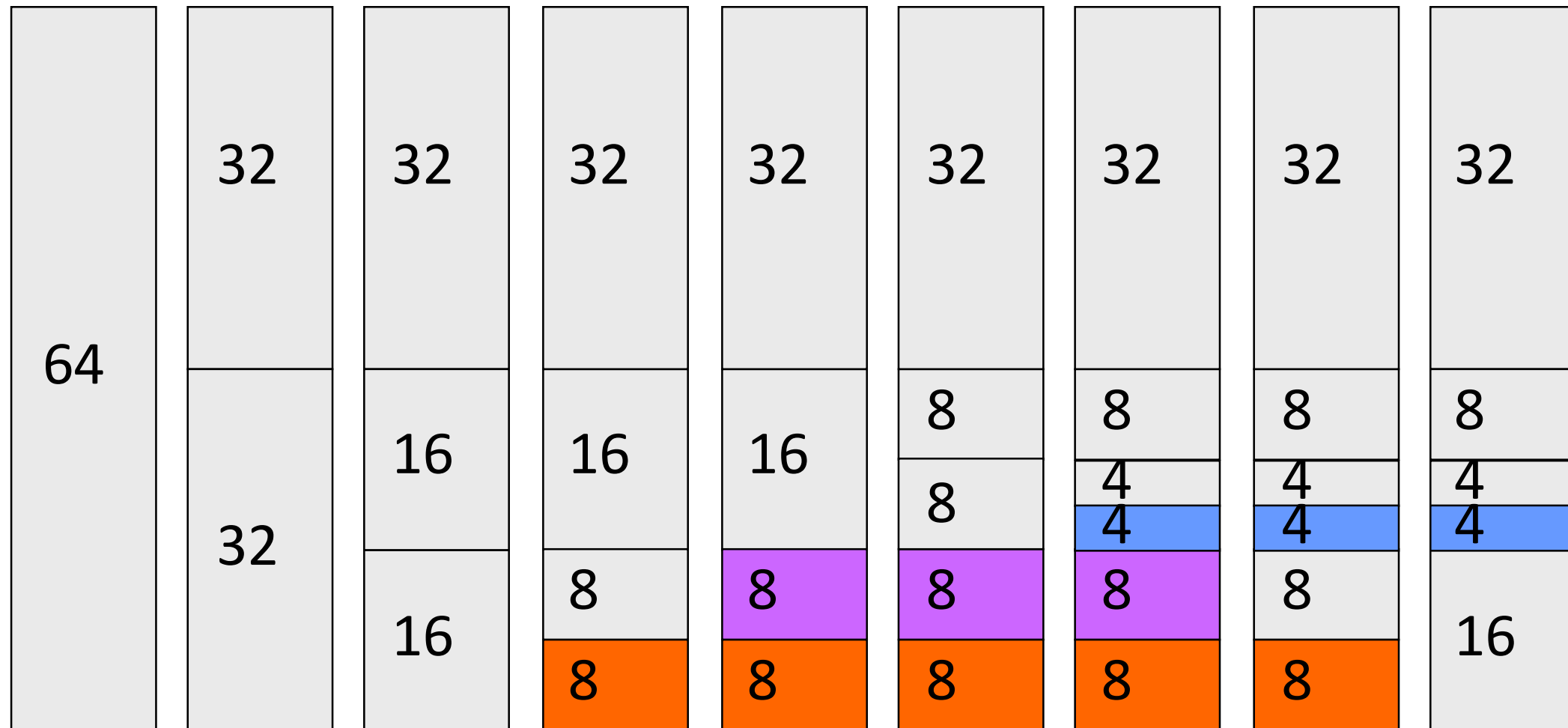
- Ad ogni richiesta di memoria (adotta strategia best-fit):
 - Sia k il numero di pagine richieste (es. $k=5$).
 - Si calcola m pari alla prima potenza di 2 superiore a k (es. $m=8$).
 - Se esiste una regione nella lista delle regioni a dimensione m , viene allocata alla richiesta.
 - Altrimenti si cerca nelle liste delle regioni di dimensioni immediatamente superiori ($2m, 4m, \dots$) fino a trovare una regione libera.
 - Sia $2^h \cdot m$ la sua dimensione, essa viene divisa in due buddy per h volte fino ad ottenere una regione della dimensione desiderata.
- **Vantaggi:**
 - Allocazione della memoria molto veloce
- **Svantaggi:**
 - Frammentazione interna ed esterna
 - Esistono dei meccanismi per riutilizzare la memoria “sprecata” a causa della frammentazione interna

Gestione della Memoria in Linux

Memoria Fisica

Algoritmo **buddy-heap** – Esempio:

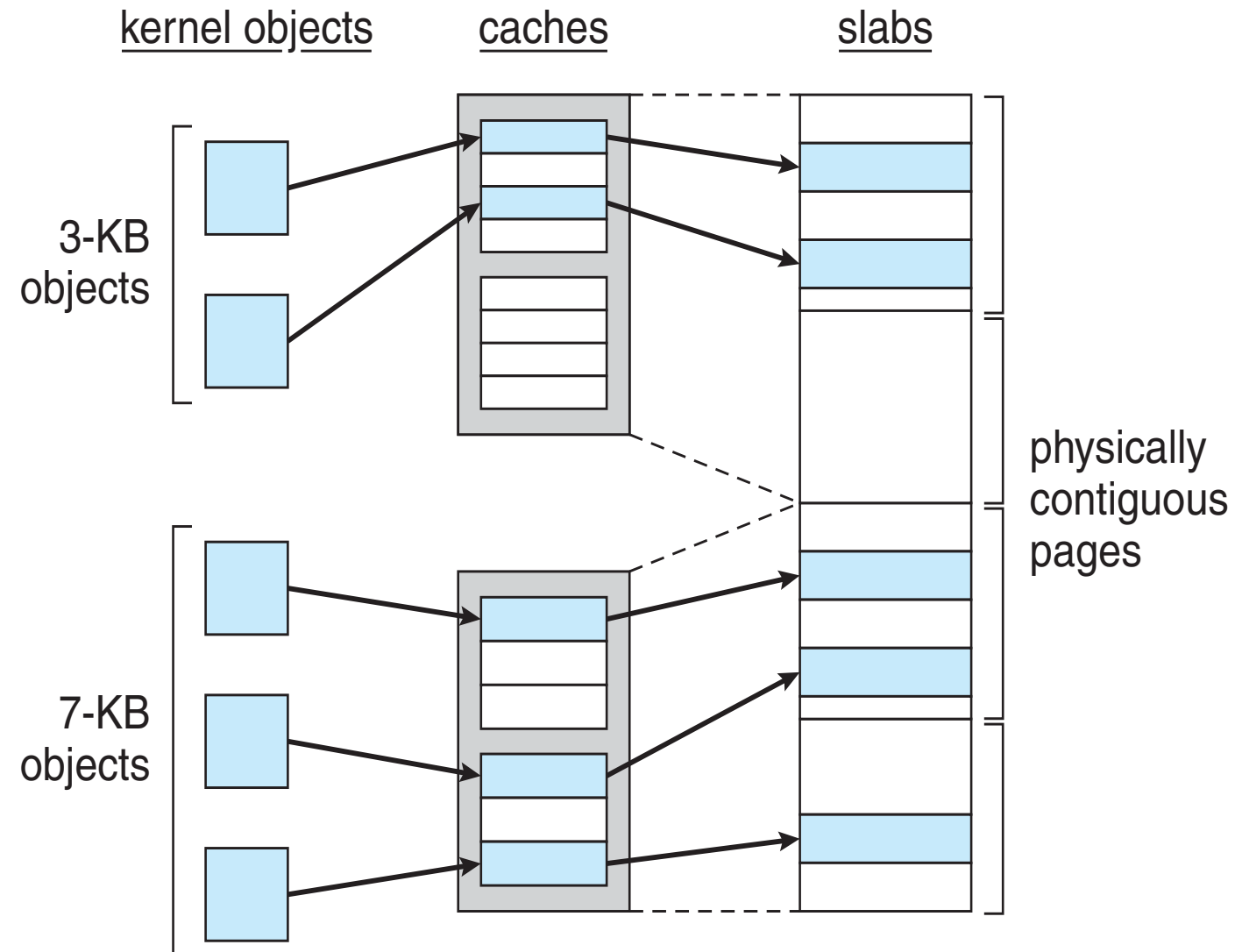
- Sequenza di richieste: **8 pagine**, **8 pagine**, **4 pagine**;
- Seguita dal rilascio: **8 pagine**, **8 pagine**



Algoritmo **slab**

- Uno **slab** è formato da una o più pagine fisicamente contigue
- Una **cache** è composta da uno o più slab
 - Ogni cache è dedicata ad un solo tipo di struttura dati del kernel
- Un **oggetto** è una istanza di una struttura dati del kernel
- Ogni cache viene riempita con oggetti (istanze della struttura dati)
 - Quando la cache è stata creata, viene riempita di oggetti contrassegnati come **liberi**
 - Quando una istanza di struttura viene memorizzata nella cache, l'oggetto che gli è stato assegnato viene contrassegnato come **usato**
 - Se uno slab è pieno di oggetti usati, l'oggetto successivo viene assegnato ad un altro slab vuoto
 - Se non ci sono slab vuoti, viene creato un nuovo slab (viene richiesta l'allocazione di memoria fisica)
- I vantaggi includono riduzione della frammentazione, velocità nell'allocare memoria agli oggetti

Algoritmo **slab** (*continua*)



Algoritmo **slab** – Esempio

- Il descrittore di processo è di tipo `struct task_struct`
 - occupa circa 1,7 KB di memoria
- Creazione di un nuovo task-> alloca nuova struct dalla cache
 - Utilizzerà una `struct task_struct` libera esistente
- Lo slab può essere in tre stati possibili
 - Full – tutto usato
 - Empty – tutto libero
 - Partial – mix di libero e usato
- Su richiesta, lo slab allocator:
 1. Utilizza una struct libera nello slab parzialmente occupato
 2. Se lo slab corrente è pieno, ne prende una libera da uno slab vuoto
 3. Se non ci sono slab vuoti, ne crea un nuovo vuoto



Gestione della Memoria in Linux

Memoria Fisica

Linux 2.2 aveva SLAB, ora ha due allocatori: SLOB e SLUB

- SLOB per sistemi con memoria limitata
- SLUB è un SLAB con prestazioni ottimali

Gestione della Memoria in Linux

Memoria Virtuale

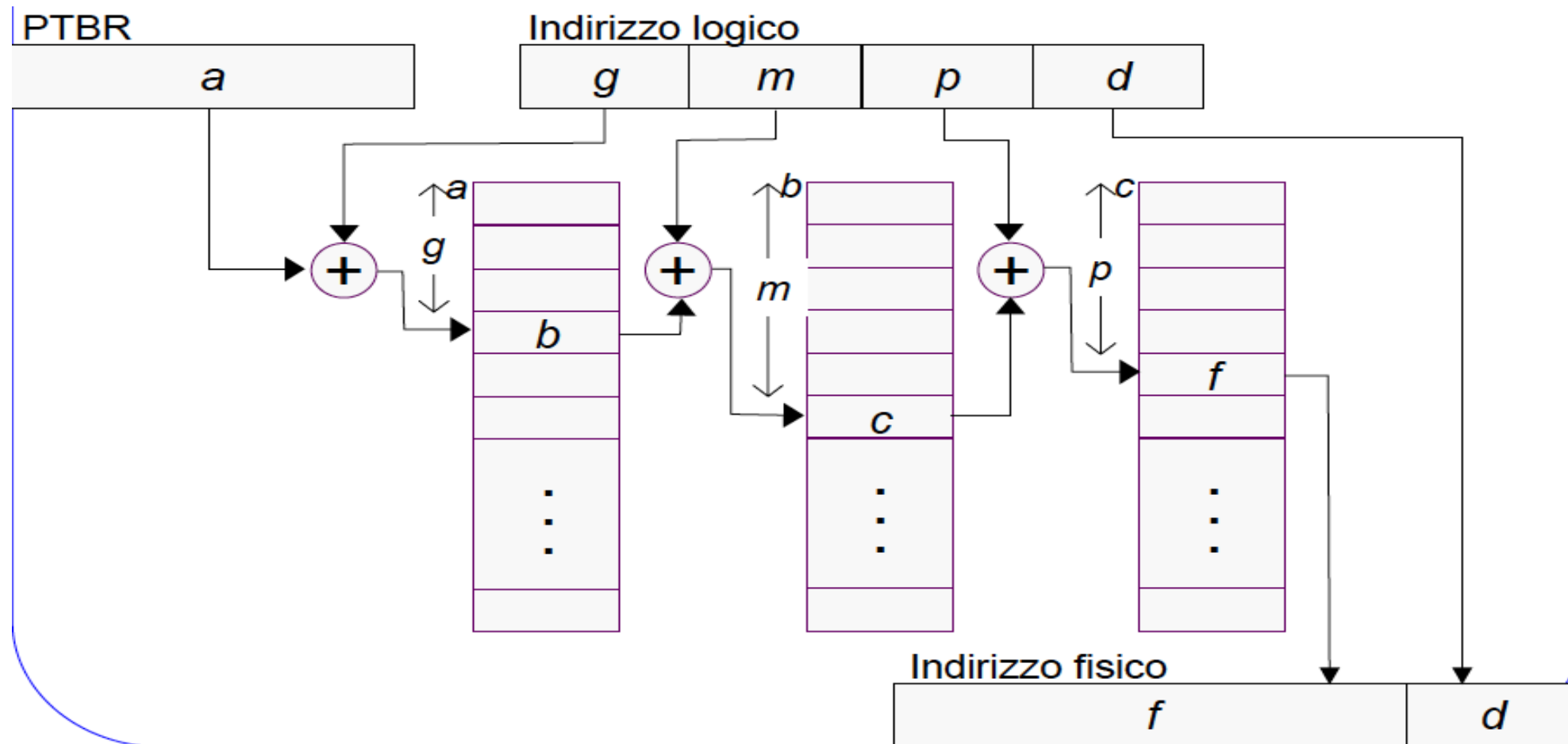
- Regioni di memoria virtuale
- Lo spazio di indirizzamento è costituito da un insieme di regioni.
- Ogni regione rappresenta un insieme di pagine logicamente contigue, contenenti informazioni omogenee tra loro (codice, dati, ...).
- Ogni regione è descritta da una struttura dati (`vm_area_struct`) che contiene tra l'altro:
 - permessi di lettura, scrittura ed esecuzione,
 - proprietà di paginazione,
 - informazioni riguardo i file,
 - numero della prima e dell'ultima pagina appartenente alla regione.
- Le regioni relative ad uno spazio di indirizzamento sono organizzate in un albero binario bilanciato
 - per una rapida ricerca della regione corrispondente ad un dato indirizzo virtuale.

Gestione della Memoria in Linux

Memoria Virtuale

Paginazione su richiesta a tre livelli

- Con i processori più semplici (per es. il pentium) il secondo livello è disabilitato.



Algoritmo di sostituzione

- esiste un demone chiamato *kswapd* il cui codice viene eseguito una volta al secondo
 - verifica che ci siano abbastanza pagine libere disponibili
 - in caso contrario, seleziona una pagina occupata e la rende libera
- le pagine di codice vengono mappate direttamente nei file eseguibili da cui derivano
- tutte le altre pagine vengono mappate
 - nella partizione di swap
 - più efficiente
 - scrittura contigua, accesso diretto al dispositivo
 - in file di swap (fino a 8 file diversi)
 - meno efficiente
 - blocchi possono essere fisicamente non contigui, accesso attraverso il file system
- algoritmo di selezione
 - decide quali pagine selezionare

Algoritmo di selezione

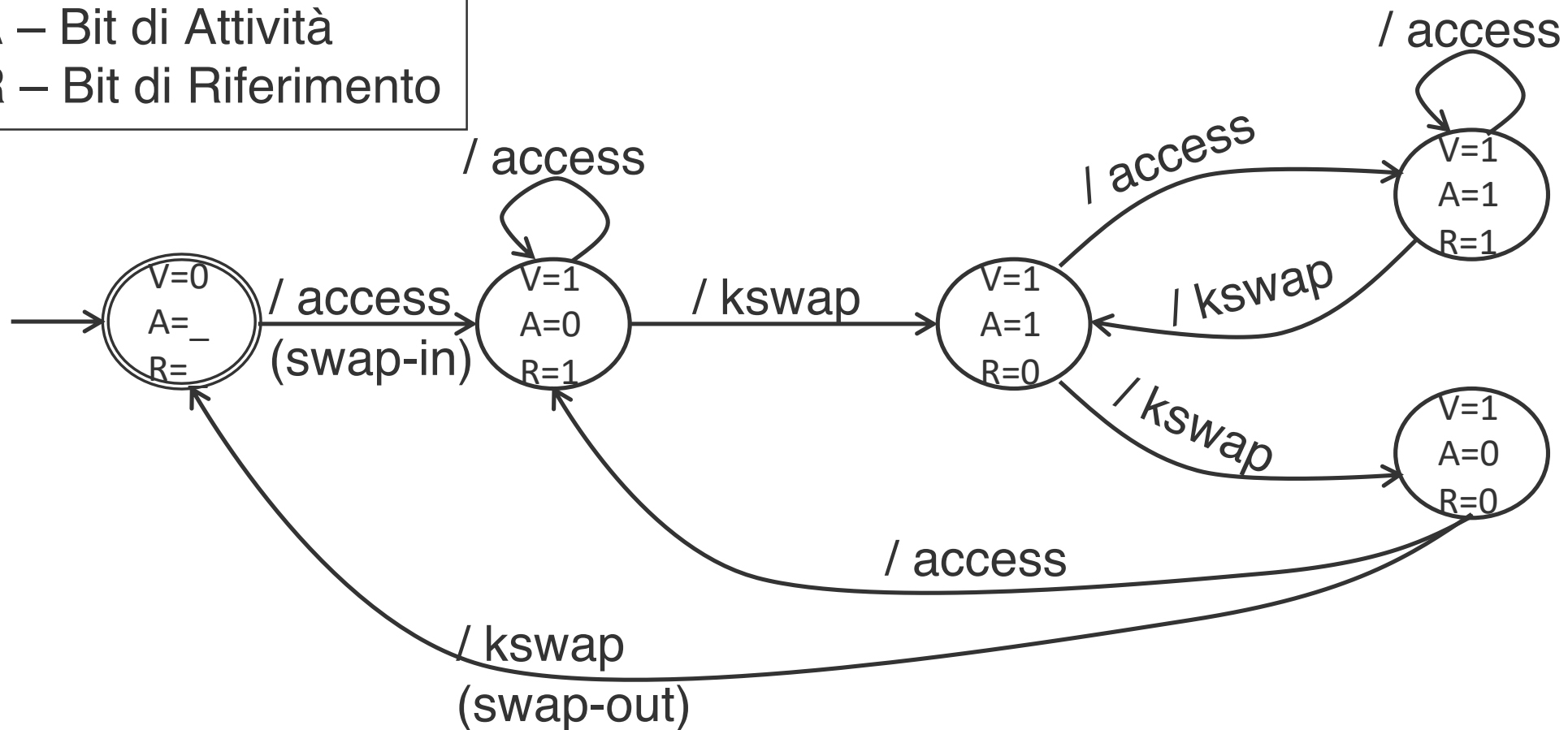
- l'algoritmo è piuttosto complesso, riportiamo una versione semplificata:
 - si cerca il processo che ha più pagine in memoria
 - si analizzano tutte le sue regioni (a partire dall'ultima regione analizzata nella ricerca precedente)
 - non vengono considerate le pagine condivise, utilizzate dai canali DMA, locked, assenti dalla memoria
 - per selezionare la pagina si adotta l'algoritmo dell'orologio a doppia scansione.

Algoritmo di selezione (*continua*)

- vengono utilizzate due liste: pagine attive e pagine inattive
- vengono utilizzati due bit:
 - Active bit: 1 – è nella lista delle pagine attive
0 – è nella lista delle pagine inattive
 - Referenced bit: 1 – è stata usata di recente
0 – non è stata usata di recente
- MMU – ogni volta che una pagina viene richiesta, mette il Referenced Bit a 1
- kswap – periodicamente controlla
 - Se Referenced Bit=1, mette Referenced Bit=0 e Active Bit=1
 - Se Referenced Bit=0 e Active Bit=1, mette Active Bit=0
- Solo le pagine inattive vengono selezionate per essere sostituite
- Lo swap-out viene fatto solo se il Dirty Bit è 1

Algoritmo di selezione (*continua*)

V – Bit di Validità
A – Bit di Attività
R – Bit di Riferimento



Gestione della Memoria in Linux

Memoria Virtuale

Algoritmo di selezione (*continua*)

